

CRAFT: COMPOSITIONAL REWARD ALIGNMENT FOR TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Loop invariants are essential to program verification, reducing unbounded loop executions to finite proof obligations. Generating strong invariants is especially difficult when verification targets are hidden: inductive validity alone does not establish sufficient strength to prove the targets, while generation lacks target guidance and training lacks target-specific proof feedback. Generated responses may contain incorrect clauses, and useful clauses needed for a proof may be scattered across responses. Inference can exploit these responses through decomposition, filtering, and composition, but whole-response training feedback may discard useful partial contributions, creating a mismatch between training and inference. We introduce CRAFT to align compositional inference, supervision targets, and reinforcement-learning rewards under *target-hidden loop verification*. At inference, CRAFT decomposes responses into clauses, pools and jointly filters them, and composes the survivors into a stronger invariant. During training, target-independent negative-trace coverage supplies a strength signal beyond inductive validity: supervised fine-tuning (SFT) learns multi-response compositions accepted by verification and coverage screening, while reinforcement learning (RL) rewards each response for the individual and shared coverage contributions of its clauses retained by the same pooled filtering procedure. Training feedback thus reflects the useful parts consumed at inference, without requiring each response to complete the proof independently. Target assertions and postconditions remain hidden throughout generation and training and are restored only for final verification. On 832 C programs, SFT raises Qwen3-8B’s `compose@1`—the verification rate after filtering a single response—from 37.86% to 63.90%, exceeding the untrained model’s 57.81% `compose@32`. Subsequent RL raises `compose@1` to 69.23%. Reward ablations show that requiring whole-response correctness can improve standalone success while degrading composition, supporting training feedback aligned with the clauses retained and used by compositional inference.

1 INTRODUCTION

Loop invariants are properties that hold at loop entry and are preserved by iteration. By summarizing states reachable after arbitrarily many iterations, they reduce unbounded execution to finite proof obligations and are central to verifying program safety and functional correctness. Yet generating sufficiently strong invariants remains difficult: a response may mix useful clauses with incorrect ones, or lack complementary constraints found in another response. A failed response can therefore contain useful proof material. Emitting more candidate clauses may lower whole-response success (pass) while expanding the candidates available for filtering and composition (compose). Our objective is the success of the filtered composition, without requiring each response to succeed independently.

We study *target-hidden loop verification*: the model sees the loop context, while target assertions and postconditions $\mathcal{G}$ are withheld. This prevents target copying and removes target-specific proof feedback from generation and training. Inductive validity alone is insufficient: a valid but weak invariant may fail to prove the hidden targets. Training therefore needs a target-independent strength signal beyond validity.

We introduce CRAFT to align compositional inference, supervision targets, and RL rewards. At inference, it decomposes responses into clauses, pools and jointly filters them, and composes the

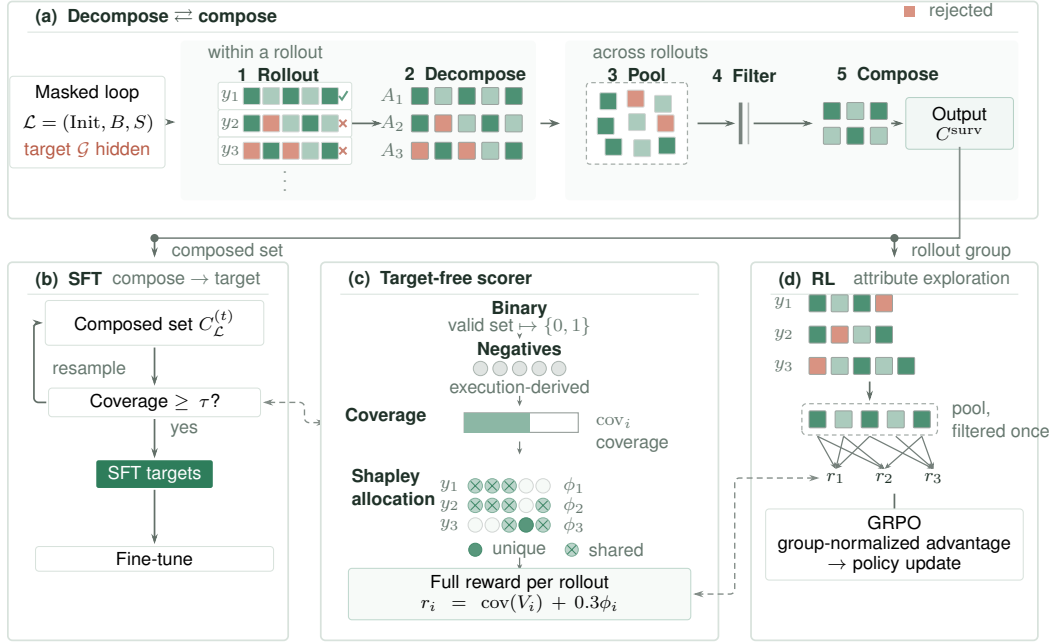


Figure 1: **Decompose-compose is the organizing principle.** Target-hidden rollouts are decomposed into clauses, so one failed clause leaves the response’s other clauses intact; clauses are pooled, filtered once per group (syntax, deduplication, Houdini), and composed into one invariant (top). SFT distills an accepted multi-response pool into one response (left). The shared target-free scorer combines execution-derived negative coverage with Shapley allocation to form Full rewards: shared rejections are split among the responses that cover them (center). RL credits each pooled survivor to its proposer rollouts and optimizes the resulting rewards with GRPO (right). The hidden target $\mathcal{G}$ is restored only for final verification.

survivors. During training, execution-derived negative-trace coverage supplies a target-independent strength signal. SFT learns multi-response compositions accepted by verification and coverage screening; RL rewards the coverage of clauses retained by the same filtering procedure, with Shapley credit allocating shared coverage among contributing responses. Useful parts can thus receive credit even when the complete response fails. The two stages aim to make multi-response discoveries accessible with fewer samples. All supervision comes from the model’s own generations, program execution, and verification, without an external teacher or reward model.

Evaluation on 832 C programs shows that compositional inference substantially improves bare-model verification and that aligned training yields further gains. We use `compose@1`, the verification rate after filtering one response, as our primary metric.

Contributions.

- We introduce a compositional inference framework for target-hidden invariant generation. Pooling clauses before Houdini filtering preserves useful parts of imperfect responses and combines complementary constraints. With one response, bare Qwen3-8B reaches 37.86% `compose@1` versus 6.43% `pass@1`.
- We align SFT targets and RL credit with compositional inference. SFT distills accepted multi-response compositions into single-response supervision, raising Qwen3-8B `compose@1` to 63.90%. RL rewards surviving clauses’ coverage and group contributions even when the complete response fails, further raising `compose@1` to 69.23%.
- Our experiments reveal that whole-response success and compositional verification can diverge. SFT broadens verification coverage and compresses the sampling budget; after substantial SFT, RL mainly improves sampling efficiency at small budgets, with little change in coverage at the largest tested budget.

2 PRELIMINARIES

Loop verification. A task contains a loop $\mathcal{L} = (\text{Init}, B, S)$ and targets $\mathcal{G} = \{(\ell, g_\ell)\}$, where Init describes loop entry, B is the guard, S is the body, and g_ℓ occurs at program location ℓ . In this paper, *valid invariant* means inductive:

$$\text{Init} \Rightarrow I, \quad \{I \wedge B\} S \{I\},$$

which implies every reachable loop-head state satisfies I (Floyd, 1967; Hoare, 1969). A finite clause set C composes as $I(C) = \bigwedge_{c \in C} c$ and is *jointly inductive* when $I(C)$ is valid. It is target-sufficient when it proves every g_ℓ at its original location; for an immediate post-loop target with only guard-false exits and a side-effect-free guard, this reduces to $I(C) \wedge \neg B \Rightarrow g$.

Targets at different locations. Let Σ be the state space and $S \subseteq \Sigma \times \Sigma$ the relation for one normally completing loop step. Write S_ℓ^B for the path-sensitive body prefix reaching location ℓ , and X_ℓ for paths from a loop head through an exit to that location, including guard-false and `break` exits. Target sufficiency requires the corresponding obligation for every $(\ell, g) \in \mathcal{G}$:

$$\begin{aligned} I(\sigma) \wedge B(\sigma) \wedge S_\ell^B(\sigma, \sigma') &\Rightarrow g(\sigma') && \text{(body target),} \\ I(\sigma) \wedge X_\ell(\sigma, \sigma') &\Rightarrow g(\sigma') && \text{(post-loop target).} \end{aligned} \quad (1)$$

Both obligations quantify universally over σ, σ' . The relations include feasible-path conditions and executable side effects; X_ℓ includes both guard-false exits and body prefixes leading to `break`; Appendix A.1 defines it explicitly. Thus joint inductiveness and target sufficiency are separate requirements: an inductive conjunction need not prove the hidden targets, and agreement with finitely many observed states does not establish induction.

Target-hidden verification. Generation and intermediate scores depend only on the visible loop context $\mathcal{L}$. Targets and target-revealing text are masked, while preconditions and executable loop semantics remain visible. The invariant is fixed before the original targets are restored for final verification. We study scalar-integer C programs with one loop; Appendix B gives the scope and an example.

3 RELATED WORK

Invariant generation and verified filtering. Classical invariant synthesis uses abstract interpretation, affine relations, templates, interpolation, syntax-guided search, or counterexamples (Cousot & Cousot, 1977; Karr, 1976; McMillan, 2003; Alur et al., 2013; Garg et al., 2014); Houdini retains an inductive subset of proposed clauses (Flanagan & Leino, 2001). Neural and LLM systems learn construction policies or combine generation with solver feedback, static analysis, filtering, and repair (Si et al., 2018; 2020; Ryan et al., 2020; Yao et al., 2020; Yu et al., 2023; Kamath et al., 2023; Chakraborty et al., 2023; Cao et al., 2025; Wu et al., 2024a; Liu et al., 2024b; Wen et al., 2024; Ma et al., 2025; Yang et al., 2026b; Wu et al., 2024b; First et al., 2023). Clause2Inv reuses clauses from previous guesses and adaptively combines them using SMT counterexamples, including postcondition-derived feedback (Cao et al., 2025). CRAFT pools clauses across independent responses before target-independent joint Houdini filtering. It couples this composition with training: accepted compositions become SFT targets, and surviving clauses determine credit for their proposing responses.

Training from formal and behavioral feedback. Verified data and execution or prover feedback have been used to train invariant, code, and proof models (Wei et al., 2025; Pinto et al., 2026; Le et al., 2022; Yan et al., 2025). Wonda curates invariant supervision from verifier output through normalization and semantic rewriting (Pinto et al., 2026). COINS and SpecRL further use positive and negative behaviors to distinguish weak valid specifications from stronger ones (Yang et al., 2026a; Huang et al., 2026); SpecRL rewards verified candidates according to the fraction of negative tests their specifications reject. CRAFT distills accepted multi-response compositions into single-response supervision and uses compositional clause-level credit to reward useful surviving parts even when the complete response fails. Its behavioral evidence comes from target-hidden executions and is applied only after clauses survive pooled verification.

Set-level credit in verifiable-reward RL. Per-response RL can improve pass@1 without expanding large- k coverage (Yue et al., 2025; He et al., 2025). Recent objectives directly optimize pass@ k or up-weight rare correct responses (Walder & Karkhanis, 2025; Chen et al., 2025; He et al., 2025). Pass@ k objectives seek at least one successful complete response. CRAFT constructs successful invariants from clauses distributed across responses, even when no individual response independently proves the target. It decomposes responses before assigning each response the coverage of its surviving clauses. Credit is computed from clauses, while the policy update uses a response-level advantage.

4 METHOD

CRAFT shares one clause-processing pipeline across inference, SFT target construction, and RL credit assignment (Figure 1). Appendix A gives the complete formal definition.

4.1 TARGET-HIDDEN COMPOSABLE INFERENCE

Clauses from different responses can provide complementary target constraints and support one another during inductive verification. We therefore pool them before filtering. Let $\text{Admit}(y)$ extract the first $m_{\max}$ invariant clauses from response y and remove duplicates, with $m_{\max}$ specified in Table 3. Algorithm 1 gives the inference procedure; $\parallel$ denotes concatenation and Unique removes duplicates.

Algorithm 1 Rollout-decompose-pool-filter-compose

Input: task $\mathcal{T} = (\mathcal{L}, \mathcal{G})$, policy π , number of responses n
Output: composed invariant I , final verdict v

- 1: $x \leftarrow \mathcal{L}$ by masking $\mathcal{G}$ in $\mathcal{T} \triangleright \text{hide target}$
- 2: $y_{1:n} \sim \pi(\cdot \mid x) \triangleright \text{rollout}$
- 3: $A_i \leftarrow \text{Admit}(y_i)$ for $i = 1, \dots, n \triangleright \text{decompose and cap}$
- 4: $P \leftarrow \text{Unique}(A_1 \parallel \dots \parallel A_n) \triangleright \text{pool in response order}$
- 5: $C \leftarrow \text{Filter}_{\mathcal{L}}(P) \triangleright \text{filter}$
- 6: $I \leftarrow \bigwedge_{c \in C} c \triangleright \text{compose}$
- 7: $v \leftarrow \text{TaskVerify}(\mathcal{T}, C) \triangleright \text{restore target}$
- 8: **return** (I, v)

Joint verification and deletion. Houdini checks each clause for establishment at loop entry and preservation under the current conjunction. It repeatedly deletes unproved clauses and rechecks the survivors, since they may have depended on removed clauses. Only a proved fixed point is returned, ensuring an inductive conjunction under a sound verifier. Budget and failure handling are specified in Appendix A.2.

Why pooling precedes filtering. Let $H_{\mathcal{L}}$ denote ideal Houdini with exact logical decisions. Filtering the union retains every clause obtainable by separately filtering the responses:

$$\bigcup_i H_{\mathcal{L}}(A_i) \subseteq H_{\mathcal{L}}\left(\bigcup_i A_i\right). \quad (2)$$

The inclusion can be strict when supporting clauses occur in different responses; its derivation is in Appendix A.10. Finite prover budgets can break this inclusion, so measured compose@ k need not be monotonic (Appendix K.6).

4.2 TARGET-INDEPENDENT STRENGTH SIGNAL

Inductive validity alone cannot distinguish useful invariants from weak ones such as true. We execute the masked loop and construct candidate negative traces $\mathcal{N}$: local relational perturbations probe constraints among variables, while continuations beyond witnessed exits probe boundary constraints. Sampling and cleaning are detailed in Appendix C.2.

For a verified clause set C , the state evaluator returns $\text{Eval}(c, s) \in \{\text{true}, \text{false}, \text{unknown}\}$. A negative trace is rejected only by a definite false evaluation:

$$\text{rej}(C) = \{\nu \in \mathcal{N} : \exists s \in \nu, \exists c \in C, \text{Eval}(c, s) = \text{false}\}. \quad (3)$$

An unknown evaluation contributes no rejection. Perturbations form singleton traces; each cleaned post-exit continuation remains one trace. The trajectory-level coverage is

$$\text{cov}(C) = \frac{|\text{rej}(C)|}{|\mathcal{N}|}, \quad |\mathcal{N}| > 0. \quad (4)$$

Each trace contributes one unit regardless of its length.

Empirical alignment. The verifier supplies soundness, while coverage provides an empirical strength signal. Final verification determines hidden-target sufficiency. Coverage achieves a within-program macro AUROC of 0.842 for final verification, supporting this association; Appendix K.5.1 gives the diagnostic population, uncertainty, and limitations.

4.3 DISTILLING MULTI-RESPONSE SEARCH

SFT distills accepted multi-response compositions into single-response supervision, aiming to make a richer set of useful clauses available with fewer samples. Starting with $P_0 = \emptyset$, each round samples g_{sft} responses from the same base model that is subsequently fine-tuned, adds their clauses to the accumulated pool, and jointly filters it:

$$P_t = \text{Unique}\left(P_{t-1} \parallel \text{Admit}(y_1^{(t)}) \parallel \dots \parallel \text{Admit}(y_{g_{\text{sft}}}^{(t)})\right), \quad C_t = \text{Filter}_{\mathcal{L}}(P_t). \quad (5)$$

Accumulating all admitted clauses in P_t allows a removed clause to return when a later response supplies support. The target can therefore combine discoveries across responses and rounds.

Acceptance and serialization. For nonempty $\mathcal{N}$, the first composition satisfying $\text{cov}(C_t) \geq \tau$ within t_{sft} rounds is serialized as the SFT target; otherwise no training pair is produced. Standard autoregressive cross-entropy trains the model on this target, conditioned on the masked loop. Algorithm 3 gives the complete procedure.

4.4 REWARDING DECOMPOSED CONTRIBUTIONS

A failed response should not erase the contributions of its useful clauses. RL therefore assigns credit to the parts retained by joint filtering. For a group sampled from the current policy, let $A_i = \text{Admit}(y_i)$. We pool and filter once, then map surviving clauses back to their proposing responses:

$$P^* = \text{Unique}(A_1 \parallel \dots \parallel A_{g_{\text{rl}}}), \quad C^* = \text{Filter}_{\mathcal{L}}(P^*), \quad (6)$$

$$V_i = \{c \in A_i : \text{key}(c) \in \text{keys}(C^*)\}, \quad R_i = \text{rej}(V_i). \quad (7)$$

Canonical keys identify which clauses each response proposed. Thus V_i retains its surviving contributions even when other clauses fail, with mutually supporting clauses checked in the pooled context used at inference.

For nonempty $\mathcal{N}$, Full, our default reward, adds Shapley credit to Clause-decomposed coverage:

$$r_i = \frac{|R_i|}{|\mathcal{N}|} + 0.3\phi_i. \quad (8)$$

With the survivor bundles fixed, let $f(\nu) = |\{j : \nu \in R_j\}|$ count the responses rejecting trace ν . The Shapley credit is

$$\phi_i = \frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i} \frac{1}{f(\nu)}. \quad (9)$$

Each rejected trace’s credit is divided among the responses covering it. At equal coverage, Full rewards responses more when their rejected traces are covered by fewer other responses, encouraging complementary effective coverage within the group. The coverage game and proof are in Appendix A.6.1; Appendix K.9 analyzes the empirical effect.

GRPO standardizes rewards within each group and applies each response’s scalar advantage to its tokens; clauses determine credit, while policy updates remain response-level. A clause that only supports another clause receives no direct coverage credit. Update details and the empty-negative fallback are in Appendix A.7 and Algorithm 4.

5 EXPERIMENTS

We organize the evaluation around four questions: how much do compositional inference and aligned training improve verification (RQ1); can CRAFT improve verification while reducing token use relative to specialized tools (RQ2); do SFT and RL contribute differently (RQ3); and which design choices enable these gains (RQ4)? Appendix J collects the complete raw results; the independent analyses are in Appendix K.

5.1 EXPERIMENTAL SETUP

Data and judge. Training data comprise 9,024 RL records and 33,346 SFT records (31,865 distinct sources) of target-masked scalar-integer single-loop programs. The raw training-program pool is drawn primarily from SV-COMP and SyGuS (Beyer, 2023; Alur et al., 2013); Appendix G describes program selection and verified SFT-target construction. SFT targets and RL rewards are produced only by the open-weight model being trained, program execution, and Frama-C/WP verification; no external teacher or closed-model supervision is used. The evaluation combines 832 programs from Code2Inv, SyGuS, SV-COMP, LIPuS, Clause2Inv, and Loopy: 316 linear, 50 nonlinear-arithmetic, and 466 Loopy instances (Si et al., 2020; Alur et al., 2013; Beyer, 2023; Yu et al., 2023; Cao et al., 2025; Kamath et al., 2023). No training program matches an evaluation program under exact, alpha-renamed, or constant-abstracted matching (Appendix F). The judge is Frama-C 27.1 (Cobalt) with the WP plug-in, Typed memory model, and Alt-Ergo/Z3 prover portfolio (Kirchner et al., 2015). Success requires it to prove induction and every restored target; all other outcomes count as zero.

Inference protocol. All CRAFT evaluation runs share target masking, the canonical prompt, clause processing, Houdini, stochastic decoding, and no re-roll or target feedback; external tools retain their own target-hidden orchestration. Each probe curve reuses one saved pool of n_{probe} responses per task. Appendix H gives decoding, serving, prover, and training details.

Metrics. Throughout evaluation, success means $\mathcal{G}$ -sufficiency under the final verifier. $\text{pass}@k$ measures standalone success with a budget of k responses; $\text{compose}@k$ measures success after filtering and composing k responses. Displayed pass results use nearest-integer-count projections of reported rates, including response-pool estimates, while compose evaluates the first k saved responses. The two metrics therefore share a nominal response budget, with different aggregation. Appendix H specifies this distinction and the reported direct-count results; Appendix L discusses sampling variance.

5.2 RQ1: HOW MUCH DO COMPOSITIONAL INFERENCE AND ALIGNED TRAINING IMPROVE VERIFICATION?

Table 1 compares four local backbones across training stages, with four API models as references, reporting pass and compose at one- and eight-response budgets.

Compositional inference alone substantially improves verification. For Bare Qwen3-8B, $\text{pass}@1$ is 6.43%, $\text{compose}@1$ reaches 37.86%, or $5.89\times$ the standalone rate. Both rates concern a one-response inference budget, showing that the framework can recover clauses sufficient for a proof from an answer that fails as a whole.

Two-stage training aligned with inference further improves verification. SFT constructs supervision from composed valid clause sets, while RL assigns credit based on the contributions of clauses retained after filtering. Qwen3-8B $\text{compose}@1$ rises from 37.86% to 63.90% after SFT and then to 69.23% after RL. All four local backbones show this progressive improvement in $\text{compose}@1$; Qwen3-4B SFT+RL reaches 67.55%, exceeding the highest API reference of 56.97% under the evaluated protocol.

Finding. Compositional inference makes better use of model outputs; supervision and rewards aligned with that inference process further improve compositional verification. Small-model competitiveness comes from the joint effect of the inference framework and aligned training.

Backbone	Training	$k = 1$		$k = 8$	
		pass	compose	pass	compose
Qwen3-4B	Bare	5.29	32.93	12.50	47.60
	SFT	40.38	62.62	59.98	76.20
	RL	1.68	48.80	4.45	62.50
	SFT+RL (CRAFT)	35.10	67.55	53.25	78.00
Qwen3-8B	Bare	6.43	37.86	17.33	54.21
	SFT	41.93	63.90	61.11	76.20
	RL	2.88	51.80	7.81	64.18
	SFT+RL (CRAFT)	30.78	69.23	47.47	76.80
Qwen3-14B	Bare	7.69	43.51	18.03	61.54
	SFT	45.67	66.35	63.46	78.37
	RL	10.46	65.40	18.75	72.20
	SFT+RL (CRAFT)	35.82	69.35	53.00	76.44
Llama 3.1-8B	Bare	0.60	27.88	3.85	51.20
	SFT	39.18	61.30	56.85	73.08
	RL	4.45	46.00	10.46	56.20
	SFT+RL (CRAFT)	35.70	61.90	53.00	74.40
<i>API models</i>					
GPT-5-nano	–	3.61	33.89	16.11	55.53
GPT-5	–	32.57	51.32	51.68	72.00
Claude Sonnet-4.6	–	36.30	56.97	44.95	66.95
DeepSeek V4-Flash	–	28.13	51.44	48.20	74.40

Table 1: Main results on all 832 programs (%). Pass evaluates complete responses; compose verifies the filtered union of candidate clauses. Pass values are nearest-integer projections of reported rates; Appendix H details the aggregation. Bold marks the highest value in each column across all rows, including API baselines. RL denotes training directly from Bare; SFT+RL starts from the SFT model. API models receive no additional training. Complete local-model curves are in Appendix J; API breakdowns are in Table 23.

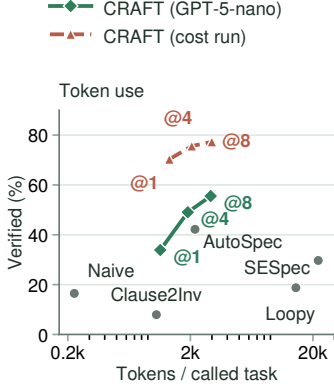


Figure 2: Verification versus tokens. LLM runs use GPT-5-nano except the local CRAFT cost run.

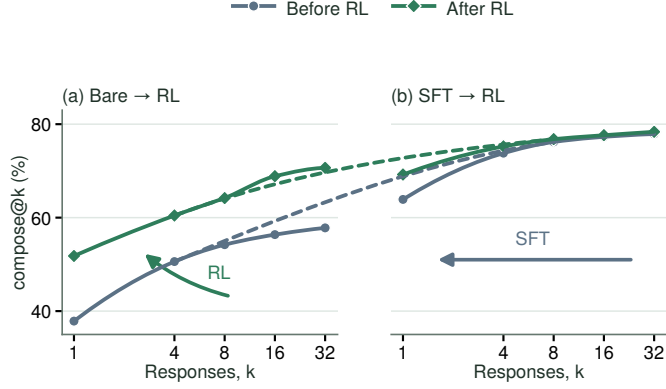


Figure 3: Qwen3-8B: RL from (a) Bare and (b) SFT. Solid curves pass through measurements; dashed guides connect interior segments. Arrows illustrate SFT budget compression and RL gains at small k .

5.3 RQ2: CAN CRAFT VERIFY MORE WHILE SPENDING FEWER TOKENS?

We compare CRAFT with AutoSpec, SESpec, Clause2Inv, Loopy, and Daikon (Wen et al., 2024; Yang et al., 2026b; Cao et al., 2025; Kamath et al., 2023; Ernst et al., 2007), using the same 832-program benchmark and judge. LLM-based tool comparisons hold GPT-5-nano and target hiding fixed.

Clause reuse improves the verification–cost frontier. With GPT-5-nano fixed, all three CRAFT budgets lie on the verification–token Pareto frontier (Figure 2). Four responses verify **49.04%** of

programs using **1,905 tokens per task**, exceeding all compared tools in verification rate while costing less than three of the four LLM-based baselines. Eight responses raise verification to 55.53% at 2,929 tokens. The locally served trained model reaches 70.31% at compose@1; detailed results appear in Appendix K.8.2.

Finding. For invariant generation with hidden verification targets and limited informative feedback, reusable clauses improve the verification–token trade-off. Compared with SESpec’s iterative repair pipeline, parallel sampling and composition verify more programs with fewer tokens.

5.4 RQ3: HOW DO SFT AND RL RESHAPE COMPOSITIONAL SEARCH?

Prior work suggests that RLVR can increase the sampling frequency of existing behaviors (Yue et al., 2025). Figure 3 compares Qwen3-8B from Bare and SFT initializations to examine whether this pattern also appears in compositional verification.

SFT expands coverage and compresses the sampling budget. SFT reaches 63.90% compose@1, exceeding Bare compose@32 (57.81%). After RL, SFT initialization raises compose@1 from 51.80% to 69.23% under the same one-response budget. It also raises compose@32 from 57.81% to 78.37%: SFT both makes useful candidates accessible with fewer responses and expands verification coverage within the measured range.

After SFT, RL mainly makes useful candidates available earlier. From Bare, RL improves compose@1 and compose@32 by 13.94 and 12.86 points, respectively, retaining a substantial gain even at the largest budget. After three SFT epochs, its gains shrink to **5.33 points at compose@1** and **only 0.47 at compose@32**. After SFT, RL mainly improves sampling efficiency: useful candidates appear with fewer responses, while coverage at the largest tested budget changes little.

Meanwhile, post-SFT RL lowers pass@1 from 41.93% to 30.78%. The independent clause-cap ablation shows the same trade-off within SFT+RL: increasing the cap from 4 to 16 lowers standalone response success while improving compose@1 and compose@8 (Appendix K.4). Both results show that whole-response success can obscure the value of useful clauses within those responses.

Finding. SFT broadens verification coverage and compresses compositional search into fewer responses. Post-SFT RL mainly improves sampling efficiency, making useful candidates available earlier even after substantial supervised training.

5.5 RQ4: WHY DO COMPOSED SFT TARGETS AND CLAUSE-LEVEL REWARDS HELP?

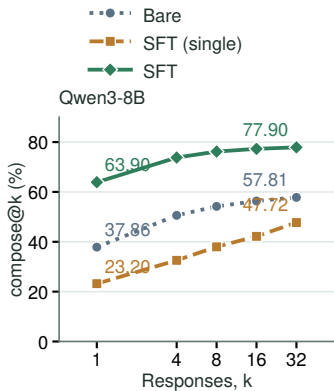


Figure 4: Qwen3-8B SFT targets and compose@k. Labels mark @1 and @32.

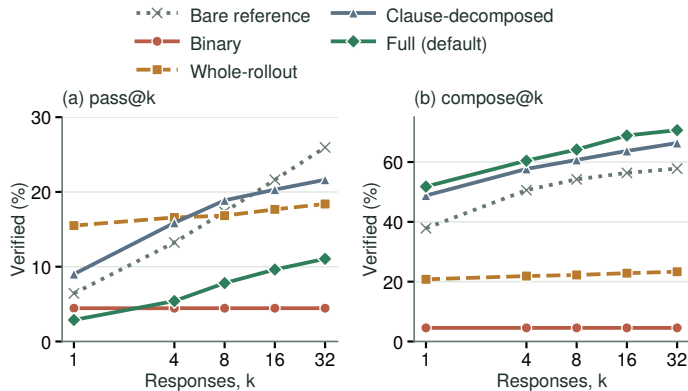


Figure 5: Qwen3-8B rewards from Bare: (a) pass@k; (b) compose@k. Panels use different y-axis ranges; parentheses mark the default.

We ablate SFT target construction and RL credit assignment to examine how training acquires useful clauses and preserves their contributions.

Composition turns broader exploration into supervision. SFT (single) uses independently filtered responses; composed targets pool valid clauses across responses, giving supervision access to more candidate invariants and complementary constraints. With approximately equal numbers of training examples, composed-target SFT raises Qwen3-8B pass@1 from 19.95% to 41.93% and compose@1 from 23.20% to 63.90%, relative to SFT (single) (Figure 4). Composition exposes SFT to the results of multi-response exploration, which the trained model can exploit with a single response.

One rejected clause should not erase the others’ credit. With verification targets hidden, RL uses inductiveness and negative coverage (Figure 5). Binary alone reaches only 4.55% compose@32. Whole-rollout adds coverage but gates credit on the entire response: pass@1 improves to 15.50%, yet compose@32 falls to 23.32%, below Bare’s 57.81%. One rejected clause therefore withholds credit from every useful survivor. Clause-decomposed removes this gate and reaches 66.35% compose@32; after SFT, its compose@1 advantage over Whole-rollout remains 24.28 points.

Group credit adds most from Bare. Full adds Shapley credit with weight 0.3. From Bare, it raises compose@32 from 66.35% to 70.67%; after SFT, the selected Full model reaches 69.23% compose@1 versus Clause-decomposed’s 67.91%, while compose@32 improves by only 0.12 points. The full comparison is in Appendix K.9.

Appendix K reports the complete ablations, including inference, clause cap, negative sampling, filtering order, and target visibility.

Finding. Composed targets turn useful clauses from broader exploration into supervision; clause-level rewards preserve useful contributions within failed responses. Together, they let training exploit more of the candidates that compositional verification can use.

6 CONCLUSION AND LIMITATIONS

Useful invariant clauses need not occur in one successful response. Cross-response filtering substantially improves bare-model verification, while SFT and RL make these gains available at smaller sampling budgets. Reward ablations show that whole-response credit can improve pass while reducing compositional verification. Projected pass estimates and prefix composition results use different aggregation at the same nominal budget, so sampling variance can affect measured gaps and saturation trends. We leave extending CRAFT beyond loop verification—to testcase generation and function contract generation—for future study (Appendix L).

REFERENCES

- Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.
- Dirk Beyer. Competition on software verification (SV-COMP). <https://sv-comp.sosy-lab.org/>, 2023.
- Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.
- Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models. *arXiv preprint arXiv:2508.10751*, 2025.
- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.
- Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, pp. 1229–1241, 2023.
- Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.
- Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pp. 19–32, 1967.
- Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.
- Andre He, Daniel Fried, and Sean Welleck. Rewarding the unlikely: Lifting GRPO beyond distribution sharpening. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.
- Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.
- Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.
- Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.

-
- Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Chang Liu, Xiwei Wu, Yuan Feng, Qinxiong Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.
- Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, and Guoqiang Li. Enhancing automated loop invariant generation for complex programs with large language models. *arXiv preprint arXiv:2412.10483*, 2024b.
- Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.
- Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.
- Ido Pinto, Yizhak Yisrael Elboher, Haoze Wu, Nina Narodytska, and Guy Katz. Not all invariants are equal: Curating training data to accelerate program verification with SLMs. *arXiv preprint arXiv:2603.15510*, 2026.
- Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.
- Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pp. 7751–7762, 2018.
- Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.
- Christian Walder and Deep Karkhanis. Pass@k policy optimization: Solving harder reinforcement learning problems. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- Anjiang Wei, Tarun Suresh, Tianran Sun, Haoze Wu, Ke Wang, and Alex Aiken. InvBench: Can LLMs accelerate program verification with invariant synthesis? *arXiv preprint arXiv:2509.21629*, 2025.
- Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.
- Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024a.
- Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024b.
- Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.
- Fanpeng Yang, Xing Li, Shuling Wang, Jie An, Zeyu Sun, Shenghua Feng, Wenhan Wang, Weiye Wang, Naijun Zhan, and Fanjiang Xu. How powerful are LLMs in generating formal program specifications? In *Proc. Int. Conf. on Machine Learning (ICML)*, 2026a.

-
- Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026b.
- Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 106–120, 2020.
- Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through SMT solving enhanced reinforcement learning. In *Proc. ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, pp. 175–187, 2023. doi: 10.1145/3597926.3598047.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

AI USE STATEMENT

Generative AI tools are integral to the evaluated system: they produce the invariant candidates and synthetic SFT data described in the paper. They were also used during research to refine hypotheses, methodology, and experiments; formulate and check mathematical claims; implement and test software; analyze and interpret experimental artifacts; prepare figures and tables; search and summarize literature; and draft, edit, and translate manuscript text. The authors reviewed all AI-assisted work and tested generated code. They take responsibility for independently validating the final content, empirical claims, and released artifacts.

REPRODUCIBILITY STATEMENT

The method and objective are specified in Section 4; Appendices C–H give the implementation defaults, prompt, training interface, data-overlap audit, and evaluation protocol. Appendix J organizes the available measurements.

A COMPLETE FORMALIZATION OF CRAFT

This section expands the method in Section 4 into a complete input-to-output specification. Given a task with hidden goals, CRAFT masks those goals and applies the same rollout–decompose–pool–filter–compose operator at inference time and during target construction. We first define the program semantics and verifier, then the exact and resource-bounded filters, and finally the inference, SFT, and RL procedures. Shapley credit and group reward collapse are formalized alongside RL scoring and policy updates; the remaining theoretical properties and proofs are collected in Appendix A.10 within this formalization.

A.1 TASK SEMANTICS, LOOP INVARIANTS, AND VERIFICATION

Let Σ be the program-state space. A loop is $\mathcal{L} = (\text{Init}, B, S)$, where Init characterizes the first loop-head states, B is the guard, and $S \subseteq \Sigma \times \Sigma$ is one normally completing loop step. Thus $S(\sigma, \sigma')$ includes execution of the body and any guard or update side effects required to reach the next loop head.

Definition 1 (Inductive invariant). *A predicate I is a valid loop invariant when*

$$\begin{aligned} \forall \sigma \in \Sigma : \text{Init}(\sigma) \Rightarrow I(\sigma), \\ \forall \sigma, \sigma' \in \Sigma : I(\sigma) \wedge B(\sigma) \wedge S(\sigma, \sigma') \Rightarrow I(\sigma'). \end{aligned} \quad (10)$$

We denote these two obligations by $\text{Ind}_{\mathcal{L}}(I)$. Here and throughout the paper, valid invariant means inductive in this sense. This implies validity on every reachable loop-head state, although the converse need not hold for the chosen assertion language and prover.

A response is decomposed into a finite candidate family C of clauses. The clauses compose by conjunction,

$$I(C) = \bigwedge_{c \in C} c, \quad I(\emptyset) = \text{true}. \quad (11)$$

The set C is *jointly inductive* when $\text{Ind}_{\mathcal{L}}(I(C))$ holds. Notice that preservation is checked under the full conjunction: a clause may rely on companion clauses. The logical definition is insensitive to clause order. The bounded verifier, however, receives a deterministic sequence $D = \langle d_1, \dots, d_m \rangle$, because generated proof dependencies can follow annotation order. Set notation below denotes the members of this sequence; deletion preserves their relative order.

A complete task is $\mathcal{T} = (\mathcal{L}, \mathcal{G})$, where $\mathcal{G}$ contains the assertions and postconditions at their original program locations. For a body target, let $S_{\ell}^B \subseteq \Sigma \times \Sigma$ be the path-sensitive prefix relation from the loop head through guard evaluation to ℓ . For a target after the loop, let E_0 be the guard-false exit relation and E_b the relation from a loop head through the body prefix to a `break` that exits this loop. These relations include guard and branch conditions and all side effects. Let $F_{e,\ell}$ be the suffix from exit e to ℓ , the identity for an immediately following target. Define the complete exit-to-target relation by

$$X_{\ell}(\sigma, \sigma') \iff \bigvee_{e \in \{0\} \cup \mathcal{B}} \exists \eta : E_e(\sigma, \eta) \wedge F_{e,\ell}(\eta, \sigma'),$$

where $\mathcal{B}$ indexes the loop’s break exits. For a side-effect-free guard, $E_0(\sigma, \eta)$ is $\neg B(\sigma) \wedge \eta = \sigma$. Unlike E_0 , a break exit may start with $B(\sigma)$ true. Then define

$$\text{VC}_{\mathcal{L}}(I, \ell, g) = \begin{cases} \forall \sigma, \sigma' : I(\sigma) \wedge B(\sigma) \wedge S_{\ell}^B(\sigma, \sigma') \Rightarrow g(\sigma'), & \ell \text{ is inside the body,} \\ \forall \sigma, \sigma' : I(\sigma) \wedge X_{\ell}(\sigma, \sigma') \Rightarrow g(\sigma'), & \ell \text{ is after the loop.} \end{cases} \quad (12)$$

Postconditions are represented by the second case, with suffix relations including assignments and return-value substitution. A return or other abrupt exit inside the body, when admitted, requires its own exit relation and applicable target obligations in the same union; it does not contribute to the normally completing transition S . For example, with `while (1) { if (q) break; ... }`, the guard-false branch is empty, but the break branch still yields a nonvacuous post-loop obligation. The ideal logical verdict is

$$\text{TaskValid}_{\models}(\mathcal{T}, C) = \text{Ind}_{\mathcal{L}}(I(C)) \wedge \bigwedge_{(\ell, g) \in \mathcal{G}} \text{VC}_{\mathcal{L}}(I(C), \ell, g). \quad (13)$$

Generation and training see only $\mathcal{L}$; $\mathcal{G}$ is restored only for Equation (13).

For a proof goal g generated by the configured loop verifier, define its portfolio status by

$$\text{GoalStatus}_\delta(g) \in \{\text{valid}, \text{timeout}, \text{unknown}\}, \quad (14)$$

where valid means that at least one configured prover returned *Valid*; timeout means that no prover proved the goal and at least one exhausted δ ; and unknown contains every other non-valid, unsupported, missing, or malformed result.

Let $\mathcal{S}_{\text{ver}} = \{\text{valid}, \text{timeout}, \text{unknown}\}$. One invocation of the resource-bounded loop verifier on the current ordered family $D = \langle d_1, \dots, d_m \rangle$ returns the clause-to-status map

$$\text{LoopVerify}_{\mathcal{L},\delta}(D) = \{d_k \mapsto v_{D,k} : 1 \leq k \leq m\} \in \mathcal{S}_{\text{ver}}^D, \quad (15)$$

where $v_{D,k} = \text{valid}$ means that the verifier proves all establishment and preservation obligations generated for d_k in the current family D . If the obligations are not all proved, $v_{D,k} = \text{timeout}$ records an exhausted prover budget and $v_{D,k} = \text{unknown}$ records every other outcome. The interface is sound when

$$(\forall c \in D : \text{LoopVerify}_{\mathcal{L},\delta}(D)[c] = \text{valid}) \implies \text{Ind}_{\mathcal{L}}(I(D)). \quad (16)$$

The filter treats timeout and unknown identically, but the interface keeps them distinct. Appendix C.1 describes how the implementation generates and aggregates the clause goals.

After the invariant set is fixed, the separate binary operator $\text{TaskVerify}_\delta(\mathcal{T}, C)$ runs the verifier on the original task annotated with C . Let $\text{TargetGoals}(\mathcal{T}, C)$ be its generated source-level assertion and postcondition goals. Then

$$\begin{aligned} \text{TaskVerify}_\delta(\mathcal{T}, C) = & \mathbf{1}[\text{SyntaxOK}(\mathcal{T}, C) \wedge \\ & \forall c \in C : \text{LoopVerify}_{\mathcal{L},\delta}(C)[c] = \text{valid} \wedge \\ & \forall g \in \text{TargetGoals}(\mathcal{T}, C) : \text{GoalStatus}_\delta(g) = \text{valid}]. \end{aligned} \quad (17)$$

A.2 IDEAL AND RESOURCE-BOUNDED HOUDINI FILTERING

We define an exact semantic fixed-point operator and its resource-bounded verifier counterpart.

For $c \in C$, define its establishment and context-dependent preservation obligations by

$$\begin{aligned} \text{Est}_{\mathcal{L}}(c) & \iff \forall \sigma : \text{Init}(\sigma) \Rightarrow c(\sigma), \\ \text{Pres}_{\mathcal{L},C}(c) & \iff \forall \sigma, \sigma' : I(C)(\sigma) \wedge B(\sigma) \wedge S(\sigma, \sigma') \Rightarrow c(\sigma'). \end{aligned} \quad (18)$$

Ideal Houdini starts with $C_0 = C$ and repeats

$$C_{j+1} = \text{F}_{\mathcal{L}}(C_j) = \{c \in C_j : \text{Est}_{\mathcal{L}}(c) \wedge \text{Pres}_{\mathcal{L},C_j}(c)\} \quad (19)$$

until a fixed point $\text{H}_{\mathcal{L}}(C)$ is reached.

The implemented deletion round uses the clause-level loop verifier and preserves the relative order of retained clauses:

$$\widehat{\text{F}}_{\mathcal{L},\delta}(D) = \langle c \in D \mid \text{LoopVerify}_{\mathcal{L},\delta}(D)[c] = \text{valid} \rangle. \quad (20)$$

Thus timeout and unknown both cause conservative deletion. Every status in one round is computed under the same pre-deletion context D . A clause removed at the end of that round may therefore have helped another clause receive valid. The survivors must be verified again without the removed hypotheses; this is why a single deletion pass is insufficient. For an ordered input sequence D , let $D_0 = D$ and $D_{j+1} = \widehat{\text{F}}_{\mathcal{L},\delta}(D_j)$. Iteration stops at the first proved fixed point or at the symbolic round limit h_{max} . Let

$$j^* = \min\{j < h_{\text{max}} : \widehat{\text{F}}_{\mathcal{L},\delta}(D_j) = D_j\}, \quad (21)$$

when this set is nonempty. The implemented filter is

$$\text{Filter}_{\mathcal{L}}^{\delta, h_{\text{max}}}(D) = \begin{cases} D_{j^*}, & j^* \text{ exists,} \\ \langle \rangle, & \text{the round limit is reached first.} \end{cases} \quad (22)$$

At a returned fixed point, every surviving clause has just been proved in the context of that same sequence. Reaching the limit returns the empty sequence. Under a sound loop verifier, the result is sound and may be smaller than the ideal fixed point. In the remainder, $\text{Filter}_{\mathcal{L}}$ abbreviates Equation (22), and TaskVerify abbreviates TaskVerify_δ .

A.3 THE SHARED COMPOSITION OPERATOR

Let $\text{parts}(y)$ extract the ordered sequence of invariant clauses in response y , let $\text{First}_m(L)$ be the prefix of sequence L containing at most m clauses, and let $\parallel$ denote sequence concatenation. Define the clauses admitted from one response by

$$\text{Admit}(y) = \text{Unique}(\text{First}_{m_{\max}}(\text{parts}(y))). \quad (23)$$

Unique removes duplicates using the finite-rule canonical normalization $\text{key}(c)$. Equal keys identify the clauses merged by those rules. Provenance is retained by recording which responses supplied each key. It keeps the first occurrence; concatenations are traversed in response order and then clause order, which fixes the sequence supplied to `LoopVerify`. For a clause sequence D , let $\text{keys}(D) = \langle \text{key}(c) : c \in D \rangle$.

For any response sequence $y_{1:n}$, define

$$A_i = \text{Admit}(y_i), \quad P(y_{1:n}) = \text{Unique}(A_1 \parallel \dots \parallel A_n), \quad (24)$$

$$C(y_{1:n}) = \text{Filter}_{\mathcal{L}}(P(y_{1:n})), \quad I(y_{1:n}) = \bigwedge_{c \in C(y_{1:n})} c. \quad (25)$$

This is the shared rollout–decompose–pool–filter–compose operator. The filter checks preservation under the complete current conjunction, so clauses from different responses may support one another; standalone preservation is not required.

A.4 COMPLETE INFERENCE ALGORITHM

Algorithm 2 CRAFT inference

Input: task $\mathcal{T} = (\mathcal{L}, \mathcal{G})$, policy π , number of responses n
Output: composed invariant I , final verdict v

- 1: $x \leftarrow \text{Mask}(\mathcal{T}) = \mathcal{L} \triangleright \text{hide goals}$
- 2: sample $y_{1:n} \sim \pi(\cdot \mid x) \triangleright \text{rollout}$
- 3: $A_i \leftarrow \text{Admit}(y_i)$ for every $i \triangleright \text{decompose and cap}$
- 4: $P \leftarrow \text{Unique}(A_1 \parallel \dots \parallel A_n) \triangleright \text{pool}$
- 5: $C \leftarrow \text{Filter}_{\mathcal{L}}^{\delta, h_{\max}}(P) \triangleright \text{filter}$
- 6: $I \leftarrow \bigwedge_{c \in C} c \triangleright \text{compose}$
- 7: $v \leftarrow \text{TaskVerify}_{\delta}(\mathcal{T}, C) \triangleright \text{restore goals}$
- 8: **return** (I, v)

For evaluation, $\text{compose}@n$ is the percentage of tasks for which $\text{TaskVerify}(\mathcal{T}, C(y_{1:n}))$ succeeds on the first n saved responses. $\text{Pass}@n$ measures standalone success at the same nominal budget; archived results estimate it over subsets of the saved pool (Appendix H).

A.5 COMPLETE SFT TARGET CONSTRUCTION

SFT distills the same multi-response map into one response. It accumulates raw candidates so that a clause discarded in one round can later acquire a supporting companion. The policy π_0 is the small open-weight model being trained and is the only learned generator in target construction. Executions of $\mathcal{L}$ produce the trace signal, and $\text{Filter}_{\mathcal{L}}$ supplies the verifier decision; no external model supplies candidates, labels, or acceptance judgments.

Algorithm 3 Multi-response SFT synthesis

Input: masked loop $\mathcal{L}$, policy π_0 , negatives $\mathcal{N}$, group size g_{sft} , round limit t_{sft}
Output: one serialized target, or reject

```

1: if  $\mathcal{N} = \emptyset$  then return reject
2:  $P \leftarrow \langle \rangle$ 
3: for  $t = 1, \dots, t_{\text{sft}}$  do
4:   sample a response group  $y_{1:g_{\text{sft}}} \sim \pi_0(\cdot \mid \mathcal{L})$ 
5:    $P \leftarrow \text{Unique}(P \parallel \text{Admit}(y_1) \parallel \dots \parallel \text{Admit}(y_{g_{\text{sft}}}))$ 
6:    $C \leftarrow \text{Filter}_{\mathcal{L}}(P)$ 
7:   if  $\text{cov}_{\mathcal{N}}(C) \geq \tau$  then return  $\text{Serialize}(C)$ 
8: return reject

```

The SFT input and target contain no element of $\mathcal{G}$. The parameter values appear in Appendix C.

A.6 COMPLETE RL GROUP-SCORING ALGORITHM

The sampled policy is the same small open-weight model updated by RL. Given its rollouts, every reward component below is computed from program executions, verifier outcomes, and deterministic clause provenance; no external model enters the scoring function.

Executions of the masked loop yield reachable loop-head states $\mathcal{E}$ and target-independent negative traces $\mathcal{N}$. The lightweight evaluator is three-valued:

$$\text{Eval}(c, s) \in \{\text{true}, \text{false}, \text{unknown}\}. \quad (26)$$

An unknown result leaves the negative trace unrejected and the candidate out of the reachable-state front end's accepted set; the verifier handles syntax and induction. A trace counts as rejected only when at least one retained clause evaluates definitively to false at one of its states:

$$\text{rej}_{\mathcal{N}}(C) = \{\nu \in \mathcal{N} : \exists s \in \nu, \exists c \in C, \text{Eval}(c, s) = \text{false}\}, \quad \text{cov}_{\mathcal{N}}(C) = \frac{|\text{rej}_{\mathcal{N}}(C)|}{|\mathcal{N}|}. \quad (27)$$

Coverage is trajectory level and is defined only for $|\mathcal{N}| > 0$. Instances without retained negatives are rejected during SFT construction. For RL, the explicit fallback below uses a binary survivor signal and never evaluates this ratio.

For an RL group $y_{1:g_{\text{rl}}}$, first form the same globally deduplicated pool used at inference, and then invoke the verifier filter once:

$$A_i = \text{Admit}(y_i), \quad P^* = \text{Unique}(A_1 \parallel \dots \parallel A_{g_{\text{rl}}}), \quad (28)$$

$$C^* = \text{Filter}_{\mathcal{L}}(P^*), \quad V_i = \langle c \in A_i \mid \text{key}(c) \in \text{keys}(C^*) \rangle, \quad R_i = \text{rej}_{\mathcal{N}}(V_i). \quad (29)$$

Here V_i assigns each pooled survivor back to every response that proposed it. A reachable-state front end evaluates the globally pooled candidates on $\mathcal{E}$ before the verifier processes the same pool. If $f(\nu) = |\{j : \nu \in R_j\}|$, the reward is

$$r_i = w_c \frac{|R_i|}{|\mathcal{N}|} + w_g \left(\frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i} \frac{1}{f(\nu)} \right). \quad (30)$$

Here $w_c = 1.0$ and $w_g = 0.3$ weight coverage and group credit. The first term rewards each response by the negative coverage of its pooled survivors. The second is the response-level Shapley value of the fixed post-filter coverage game $u(A) = |\bigcup_{i \in A} R_i|/|\mathcal{N}|$: repeated coverage is shared among its proposers, while unique coverage receives full credit. Thus $1/f(\nu)$ is a group-local difficulty weight: negatives rejected by many responses receive less credit per response than rare negatives rejected by few. The negatives are nonuniformly weighted for each response, while their credits still sum to one unit per covered trace across the group. A purely supporting clause may enable another clause to survive but receives no direct coverage credit. The per-response cap limits admission without penalizing excess output.

Algorithm 4 CRAFT reward for one RL group

Input: masked loop $\mathcal{L}$, responses $y_{1:g_{rl}}$, reachable states $\mathcal{E}$, negative traces $\mathcal{N}$
Output: response rewards $r_{1:g_{rl}}$
1: $A_i \leftarrow \text{Admit}(y_i)$ for every i
2: $P^* \leftarrow \text{Unique}(A_1 \parallel \dots \parallel A_{g_{rl}}) \triangleright \text{pool globally}$
3: $C^* \leftarrow \text{Filter}_{\mathcal{L}}(P^*) \triangleright \text{filter the pool once}$
4: $V_i \leftarrow \langle c \in A_i \mid \text{key}(c) \in \text{keys}(C^*) \rangle$ for every $i \triangleright \text{project by provenance}$
5: **if** $\mathcal{N} = \emptyset$ **then**
6: $r_i \leftarrow \mathbf{1}[A_i \neq \langle \rangle \wedge \text{keys}(V_i) = \text{keys}(A_i)]$ for every i
7: **return** $r_{1:g_{rl}}$
8: $R_i \leftarrow \text{rej}_{\mathcal{N}}(V_i)$ for every i
9: $f(\nu) \leftarrow |\{j : \nu \in R_j\}|$ for every $\nu \in \mathcal{N}$
10: $r_i \leftarrow w_c |R_i| / |\mathcal{N}| + w_g |\mathcal{N}|^{-1} \sum_{\nu \in R_i} f(\nu)^{-1}$ for every i
11: **return** $r_{1:g_{rl}}$

A clause c *semantically supports* d in D when $c \neq d$, $\text{Pres}_{\mathcal{L},D}(d)$ holds, but $\text{Pres}_{\mathcal{L},D \setminus \{c\}}(d)$ does not. The implemented verifier exposes the broader, resource-dependent relation

$$\text{VSupport}_{\mathcal{L},\delta}(c, d \mid D) \iff c \neq d \wedge \text{LoopVerify}_{\mathcal{L},\delta}(D)[d] = \text{valid} \wedge \text{LoopVerify}_{\mathcal{L},\delta}(D \setminus \{c\})[d] \neq \text{valid}. \quad (31)$$

This relation includes semantic support, generated-goal dependencies, and resource effects. Algorithm 4 can retain such a clause through joint filtering, but assigns it direct reward only if its projected response rejects a negative trace.

A.6.1 SHAPLEY CREDIT: DEFINITION AND EQUIVALENCE

Fix the survivor bundles from one pooled filtering call and write $R_i = \text{rej}_{\mathcal{N}}(V_i)$ and $f(\nu) = |\{j : \nu \in R_j\}|$. For nonempty $\mathcal{N}$,

$$r_i^{\text{Clause-decomposed}} = \frac{|R_i|}{|\mathcal{N}|}, \quad \phi_i = \frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i} \frac{1}{f(\nu)}, \quad (32)$$

$$r_i^{\text{Full}} = r_i^{\text{Clause-decomposed}} + 0.3\phi_i.$$

Equivalence to the standard Shapley value. Equation (32) gives a closed form for the standard response-level Shapley value. For response i , let V_i be its projected bundle of clauses that survive filtering of the full response group, and let $R_i = \text{rej}_{\mathcal{N}}(V_i)$. A coalition S contributes the projected bundles of its responses, whose rejected-trace set is $\bigcup_{i \in S} R_i$.

Proposition 2 (Equivalence to standard Shapley). *Let $G = \{1, \dots, g\}$ index the responses above, and define their coverage game by*

$$u(S) = \frac{1}{|\mathcal{N}|} \left| \bigcup_{j \in S} R_j \right|, \quad S \subseteq G.$$

The standard Shapley value of response i ,

$$\text{Sh}_i(u) = \sum_{S \subseteq G \setminus \{i\}} \frac{|S|! (g - |S| - 1)!}{g!} (u(S \cup \{i\}) - u(S)), \quad (33)$$

is exactly

$$\text{Sh}_i(u) = \frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i} \frac{1}{f(\nu)} = \phi_i, \quad f(\nu) = |\{j : \nu \in R_j\}|.$$

Consequently, $\sum_{i \in G} \phi_i = u(G)$.

Proof. The coefficient in Equation (33) is the fraction of permutations of G in which exactly the members of S precede i : there are $|S|!$ orders before i , $(g - |S| - 1)!$ orders after it, and $g!$ orders in total. Hence the standard subset definition equals the expected marginal contribution of response i under a uniformly random permutation.

For each trace ν , define $Q_\nu = \{j \in G : \nu \in R_j\}$. If $Q_\nu = \emptyset$, the trace never contributes to the game. Otherwise, it changes the coalition value exactly once in any permutation: when the first response from Q_ν is added. By symmetry, each member of Q_ν is first with probability $1/|Q_\nu| = 1/f(\nu)$. Thus trace ν contributes $1/(|\mathcal{N}|f(\nu))$ to $\text{Sh}_i(u)$ when $\nu \in R_i$, and zero otherwise. Summing these contributions over traces gives Equation (32). Finally, every trace covered by the grand coalition allocates a total of $f(\nu) \cdot 1/(|\mathcal{N}|f(\nu)) = 1/|\mathcal{N}|$, proving efficiency. $\square$

This equivalence is evaluated on the fixed response-level bundles $V_1, \dots, V_g$ projected from the globally filtered survivor set. It therefore proves that the implemented allocation is the standard Shapley value of this response-level coverage game. It assigns credit to responses using fixed post-filter survivor bundles.

A.7 GROUP-RELATIVE POLICY UPDATE

The rollout policy $\pi_{\theta_{\text{old}}}$ samples a group, which is scored by Algorithm 4. Define

$$\hat{A}_i = \frac{r_i - \text{mean}_j r_j}{\text{std}_j r_j + \delta_{\text{adv}}}. \quad (34)$$

For token $a_{i,u}$ and its prefix $h_{i,u} = (\mathcal{L}, a_{i,<u})$, let

$$\rho_{i,u}(\theta) = \frac{\pi_\theta(a_{i,u} | h_{i,u})}{\pi_{\theta_{\text{old}}}(a_{i,u} | h_{i,u})}, \quad (35)$$

$$q_{i,u}(\theta) = \frac{\pi_{\text{ref}}(a_{i,u} | h_{i,u})}{\pi_\theta(a_{i,u} | h_{i,u})}, \quad k_{i,u}(\theta) = q_{i,u} - \log q_{i,u} - 1. \quad (36)$$

The nonnegative quantity $k_{i,u}$ is the sampled conditional-token KL surrogate used by the implementation. Its expectation equals $D_{\text{KL}}(\pi_\theta(\cdot | h_{i,u}) || \pi_{\text{ref}}(\cdot | h_{i,u}))$ when the token is sampled from $\pi_\theta(\cdot | h_{i,u})$. During an update, however, the saved token comes from $\pi_{\theta_{\text{old}}}$, so $k_{i,u}$ is used as a sampled penalty under the saved old-policy tokens. The clipped group objective is

$$\mathcal{J}(\theta) = \mathbb{E} \left[\frac{1}{g_{\text{rl}}} \sum_i \frac{1}{T_i} \sum_{u=1}^{T_i} \left(\min \left\{ \rho_{i,u} \hat{A}_i, \text{clip}(\rho_{i,u}, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right\} - \beta k_{i,u} \right) \right]. \quad (37)$$

After optimizing this objective, the updated policy samples the next group.

A.7.1 GROUP REWARD COLLAPSE AND SHAPLEY TIE-BREAKING

Here *group reward collapse* means that all rewards in one sampled GRPO group are equal. Equation (34) then gives zero relative advantage to every response, so that group supplies no reward-driven policy-gradient signal; a separate KL term can still act. This condition concerns reward discrimination and does not require the responses themselves to be identical.

If $r_i^{\text{Clause-decomposed}} = c$ for every response in a group, then

$$r_i^{\text{Full}} - \bar{r}^{\text{Full}} = 0.3(\phi_i - \bar{\phi}), \quad \text{Var}_i(r_i^{\text{Full}}) = 0.09 \text{Var}_i(\phi_i).$$

Thus Full restores a nonzero relative signal whenever equal-coverage responses have different Shapley credits. It can distinguish which responses contribute less redundant coverage after Clause-decomposed has lost that ranking signal.

For an illustrative calculation, let $\mathcal{N} = \{a, b, c\}$ and three responses reject $R_1 = \{a, b\}$, $R_2 = \{a, c\}$, and $R_3 = \{a, c\}$, respectively. All three Clause-decomposed rewards equal $2/3$, but the trace frequencies are $f(a) = 3$, $f(b) = 1$, and $f(c) = 2$. Their Shapley credits are $(4/9, 5/18, 5/18)$, giving Full rewards $(0.80, 0.75, 0.75)$. Full therefore supplies positive relative credit to the response that uniquely rejects b . This is a worked example of the reward mechanism. It is illustrative and does not report a training outcome.

Full does not guarantee a noncollapsed group. If every response rejects the same trace set, both Clause-decomposed coverage and Shapley credit remain equal; all-zero coverage also remains uninformative. The mechanism can resolve some coverage-score ties, but establishing a lower collapse frequency in training requires logged groups from the two policies.

A.8 TRAINING–INFERENCE ALIGNMENT

Inference, SFT target construction, and RL group scoring all call the same pooled map in Equation (25); they differ only in how the returned clauses are consumed. Inference checks the restored goals, SFT serializes an accepted fixed point, and RL projects the fixed point back to the responses that proposed its surviving clauses. Negative coverage supplies the target-independent training surrogate; restored-goal verification supplies the final verdict.

A.9 EXTENSION TO VERIFIABLY COMPOSABLE GENERATION

Definition 3 (Verifiably composable generation). *A task provides visible input x , hidden goals $\mathcal{G}$, an atomic decomposition parts_x , a duplicate-removal operator Unique_x , a subset filter Filter_x , a composition operator Compose_x , and a validity predicate Valid_x . For responses $y_{1:n}$, define*

$$P_x(y_{1:n}) = \text{Unique}_x\left(\bigcup_i \text{parts}_x(y_i)\right), \quad (38)$$

$$O_x(y_{1:n}) = \text{Compose}_x(\text{Filter}_x(P_x(y_{1:n}))). \quad (39)$$

The task is verifiably composable when the filter is conservative and sound: it returns atoms from its input and $\text{Valid}_x(O_x(y_{1:n}))$ holds whenever the filter succeeds. It is target hidden when every operator above depends only on x ; the goals $\mathcal{G}$ are used only by a final task-specific check. Complementary composition occurs when $O_x(y_{1:n})$ passes that final check although no individual $O_x(y_i)$ does.

Loop-invariant generation instantiates the atoms with invariant clauses, composition with conjunction, validity with joint inductiveness, and the filter with bounded Houdini. The transfer property for any other instantiation satisfying Definition 3 is proved in Appendix A.10.

Definition 3 identifies the conditions needed to reuse CRAFT: responses must decompose into reusable atoms, atoms must admit pooling, a sound domain-specific filter must validate the pooled result, and provenance must map accepted atoms back to responses. Under these conditions, the same inference and training construction applies after replacing the invariant clauses, conjunction, and Houdini checks by the domain’s atoms, composition operator, and verifier.

For *specification generation*, atoms can be precondition, postcondition, frame, or invariant clauses; composition is conjunction; and the filter checks syntax, consistency, and the corresponding proof obligations. Separate responses can contribute complementary clauses to one verified contract. For *testcase generation*, atoms are executable tests; composition is set union; the filter removes invalid or duplicate tests; and structural or behavioral coverage supplies a target-independent signal. Separate responses can cover disjoint behaviors even when no one response forms an adequate suite.

These domains share the formal interface and algorithms. Each domain supplies its verifier and measurable training signal.

A.10 THEORETICAL PROPERTIES AND PROOFS

This subsection states and proves the properties of the algorithms defined above. Write $\mathcal{L} = (\text{Init}, B, S)$.

A.10.1 CONJUNCTION OF VALID INVARIANTS

Proposition 4 (Closure under conjunction). *If $I_1, \dots, I_m$ are valid invariants of the same loop, then $\bigwedge_j I_j$ is valid and is at least as strong as every I_j .*

Proof of Proposition 4. For each j , validity gives $\text{Init} \Rightarrow I_j$; hence $\text{Init} \Rightarrow \bigwedge_j I_j = I$. Suppose $I \wedge B$ holds before one execution of S . Then $I_j \wedge B$ holds for every j , so preservation of I_j establishes every I_j afterward. Therefore I is preserved and is a valid invariant. $\square$

Because a conjunction implies each conjunct, I is at least as strong as every I_j . $\square$

A.10.2 MUTUAL SUPPORT AMONG ARBITRARILY MANY CLAUSES

Proposition 5 (Mutual support). *For a finite clause set C , $I(C)$ is valid if*

$$\text{Init} \Rightarrow I(C), \quad \forall c \in C : \text{Pres}_{\mathcal{L}, C}(c). \quad (40)$$

No clause must be preserved in isolation.

Proof of Proposition 5. The first premise is exactly establishment of $I(C)$. For preservation, assume $I(C) \wedge B$ before executing S . The second premise establishes each c_j afterward. Since every conjunct then holds in the same successor state, their conjunction $I(C)$ also holds. Thus $\{I(C) \wedge B\} S \{I(C)\}$, and $I(C)$ is valid.

No standalone preservation premise appears in this argument. A clause may use all other conjuncts in the pre-state, so clauses that fail preservation separately may still be preserved jointly. Establishment is different: $\text{Init} \Rightarrow I(C)$ implies $\text{Init} \Rightarrow c_j$ for every j . Hence a candidate false at some loop-entry state cannot be rescued by conjunction. $\square$

A.10.3 GREATEST JOINTLY INDUCTIVE CANDIDATE SUBSET

Let $C_0 = C$, and let $C_{j+1} \subseteq C_j$ be the set remaining after one ideal Houdini deletion round in Equation (19). Because C is finite, this descending sequence reaches the fixed point $H_{\mathcal{L}}(C)$.

Theorem 6 (Candidate-relative optimality). *With exact logical decisions, the Houdini fixed point $H_{\mathcal{L}}(C)$ is jointly inductive and contains every jointly inductive subset of C . Its conjunction is therefore the strongest invariant obtainable by selecting clauses from this pool.*

This candidate-relative property applies to ideal Houdini with the supplied candidate pool. Bounded filtering may discard a valid clause after timeout. Joint checking permits mutual support in preservation, but cannot rescue a clause false at loop entry.

Proof of Theorem 6. At the fixed point, every $c \in H_{\mathcal{L}}(C)$ satisfies establishment and preservation under the full conjunction $I(H_{\mathcal{L}}(C))$. Applying Proposition 5 shows that this conjunction is valid.

It remains to prove greatestness. Let $C' \subseteq C$ be any jointly inductive subset. We show by induction over deletion rounds that $C' \subseteq C_j$. The base case $C' \subseteq C_0 = C$ is immediate. Assume $C' \subseteq C_j$, and take any $c \in C'$. Joint establishment of C' gives $\text{Init} \Rightarrow c$, so c cannot be deleted for establishment. Joint preservation gives

$$\{I(C') \wedge B\} S \{c\}.$$

Since $C' \subseteq C_j$, the antecedent $I(C_j) \wedge B$ implies $I(C') \wedge B$. Strengthening a valid Hoare precondition preserves validity, so

$$\{I(C_j) \wedge B\} S \{c\}$$

also holds. Therefore no member of C' is deleted in round j , proving $C' \subseteq C_{j+1}$. At termination, $C' \subseteq H_{\mathcal{L}}(C)$. Because conjunction reverses set inclusion, $I(H_{\mathcal{L}}(C))$ implies $I(C')$. $\square$

Pooling before filtering. Each separately filtered set $H_{\mathcal{L}}(A_i)$ is jointly inductive. By closure under conjunction, their union is a jointly inductive subset of $\bigcup_i A_i$. Theorem 6 therefore gives Equation (2). The inclusion can be strict when a clause requires a supporting hypothesis from another response. This ideal property does not ensure empirical monotonicity under finite budgets: pooling changes proof obligations and may trigger timeouts (Appendix K.6).

A.10.4 SOUNDNESS OF RESOURCE-BOUNDED HOUDINI

Proposition 7 (Bounded-filter soundness). *Assume $\text{LoopVerify}_{\mathcal{L},\delta}$ is sound. If $\text{Filter}_{\mathcal{L}}^{\delta, h_{\max}}(C)$ returns a proved fixed point D , then $D \subseteq C$ and $I(D)$ is inductive. Timeout, unknown, and exhaustion of the round budget cannot cause an unproved clause set to be returned.*

Proof. Every deletion round in Equation (20) selects a subset of its input, so $D \subseteq C$. At a returned fixed point, every clause maps to valid in that same family. Verifier soundness in Equation (16) therefore gives $\text{Ind}_{\mathcal{L}}(I(D))$. A timeout or unknown removes the affected clause, and reaching $h_{\max}$ returns the empty result rather than an intermediate set by Equation (22). $\square$

A.10.5 SUPPORTING CLAUSES AND CREDIT

Proposition 8 (Coverage credit omits pure support). *Suppose response i contributes a retained clause c that supports another retained clause in the sense of Appendix A, but $\text{rej}_{\mathcal{N}}(V_i) = \emptyset$. Then the coverage term assigned to response i is zero, irrespective of whether removing c would prevent the other clause from surviving.*

Proof. The premise gives $R_i = \emptyset$. Therefore $|R_i| = 0$, so the coverage term in Equation (30) is zero. $\square$

This formal edge case has a negligible aggregate effect in our evaluation. Post-filtering permits supporting clauses from different responses to survive jointly. Pre-filtering removes clauses that cannot survive within their own response. Under the longer verifier budget, their $\text{compose}@k$ difference ranges from -0.2 to $+0.6$ percentage points across the reported budgets (Table 21). Thus, although the post-filter coverage score does not credit pure indirect support, such cross-response dependencies have negligible aggregate effect on verification in this benchmark.

A.10.6 TRANSFER TO VERIFIABLY COMPOSABLE PROBLEMS

Proposition 9 (Composable transfer). *For any instantiation satisfying Definition 3, the pooled output $O_x(y_{1:n})$ contains only atoms drawn from the responses and satisfies Valid_x whenever filtering succeeds. Moreover, its construction is independent of the hidden goals $\mathcal{G}$.*

Proof. Decomposition and union draw atoms only from $y_{1:n}$; Unique_x removes candidates and the conservative filter returns a subset, establishing provenance to the responses. Filter soundness gives $\text{Valid}_x(O_x(y_{1:n}))$. Finally, every operator in Equation (39) is a function of x and the responses, not $\mathcal{G}$, so hidden goals enter only the final check. $\square$

B SCOPE AND MOTIVATING EXAMPLE

We focus on numerical loops because their arithmetic equalities and inequalities, including polynomial relations, admit scalable automatic checking. The study considers one braced while loop over scalar C integers, including nonlinear updates. Even this setting can hide nonlinear relations across coupled recurrences. For example:

```
1 void f(int k, int z) {
2   int x = 1, y = 1, c = 1;
3   while (c < k) { c++; x = x*z + 1; y = y*z; }
4   /*@ assert 1+x*z-x-z*y == 0; */
5 }
```

Case study: target-hidden invariant discovery. If the assertion is visible, it can be copied as a loop-invariant candidate. It holds initially because $x = y = 1$. Writing $I(x, y) = 1 + xz - x - zy$, one loop iteration gives

$$I(xz + 1, yz) = 1 + (xz + 1)z - (xz + 1) - z(yz) = zI(x, y),$$

so $I = 0$ is inductive and discharges the exit assertion. Under target hiding, the model sees the initialization and recurrences while the assertion is withheld, and must recover the relation. Archived rollout pools recover three algebraically equivalent forms:

Model	First verified prefix	Rediscovered invariant clause
GPT-5	compose@1	$x - 1 = z(x - y)$
DeepSeek V4-Flash	compose@4	$x(z - 1) = zy - 1$
Claude Sonnet-4.6	compose@8	$x(z - 1) = yz - 1$

All three clauses rearrange to the hidden assertion. None of the ten raw responses in these pools passes as a whole on this program; pooled filtering removes incompatible siblings and retains the useful relation.

Case study: composition without cross-response support. To separate target-proof composition from both sampling a better standalone response and mutual support during filtering, we replay the first four archived GPT-5-nano responses for the following task. We apply the target-hidden $\text{Filter}_{\mathcal{L}}$ used in evaluation first to each response separately and then, following the pooled order of Appendix K.6, once to their merged clause union. The restored-target judge is applied only after filtering. The assertion is shown for clarity but is hidden from every response.

```

1 /*@ requires n >= 0; */
2 void f(int n) {
3   int x = n, y = 0;
4   while (x > 0) {
5     y = y + 1;
6     x = x - 1;
7   }
8   /*@ assert y == n; */
9 }

```

Table 2 reports both the relevant admitted proposal from each response and the result of filtering that response alone.

Response	Relevant proposal	Independent survivors	Target
1	$x \geq 0, y \geq 0$	$x \geq 0, y \geq 0$	fail
2	—	$\emptyset$	fail
3	$x = n - y, y = n - x$	$x = n - y, y = n - x$	fail
4	$x \geq 0, y \geq 0$	$x \geq 0, y \geq 0$	fail
Pooled 1–4	$x \geq 0, y \geq 0, x = n - y, y = n - x$		verified

Table 2: A cross-response composition case without mutual support: every pooled survivor already survives independent filtering, yet no response alone verifies the target.

Each of the four distinct pooled clauses also survives when filtered alone, so pooling does not rescue any clause by supplying a missing inductive hypothesis. The complete pooled annotation is

```

1 /*@
2   loop invariant x >= 0;
3   loop invariant y >= 0;
4   loop invariant x == n - y;
5   loop invariant y == n - x;
6 */

```

The proof-minimal cross-response core is $x \geq 0$ from Response 1 (also duplicated by Response 4) and $y = n - x$ from Response 3. Each clause alone fails the restored-target judge. Together, the exit condition gives $x \leq 0$, the bound gives $x = 0$, and the relation yields $y = n$. The gain comes from combining complementary clauses to prove the target.

Case study: composition through cross-response support. The preceding case needs composition only to prove the target. A second archived GPT-5-nano replay shows the stronger mechanism from Proposition 5: pooling can also make a clause provably inductive. As before, the assertion is restored only after target-hidden filtering.

```

1 extern int unknown(void);
2 void f(int n) {
3     int x = 1, m = 1;
4     while (x < n) {
5         if (unknown()) m = x;
6         x = x + 1;
7     }
8     if (n > 1) { /*@ assert m >= 1; */ }
9 }

```

Replaying its first four responses gives:

Response	Relevant proposal	Independent survivors	Target
1	$m \leq x$	$m \leq x$	fail
2	$x \geq 1$	$x \geq 1$	fail
3	—	$\emptyset$	fail
4	$m \geq 1$	$\emptyset$	fail
Pooled 1–4 $m \leq x, x \geq 1, m \geq 1$			verified

Response 4’s $m \geq 1$ cannot be established inductive alone: after the nondeterministic assignment $m = x$, preservation requires $x \geq 1$. Response 2 supplies that hypothesis, so pooled filtering retains both clauses. The complete composed annotation is

```

1 /*@
2   loop invariant m <= x;
3   loop invariant x >= 1;
4   loop invariant m >= 1;
5 */

```

All four independently filtered responses fail. The pooled annotation verifies through cross-response support during filtering.

C IMPLEMENTATION DETAILS

Table 3 lists the implemented defaults. SFT synthesis, RL, and inference share Algorithm 1. RL filters the union once, then uses clause provenance to assign survivors back to their proposing rollouts before computing credit (Appendix K.6).

Symbol	Value	Role
r_{exec}	12	requested input executions; doubled for oracle-driven bodies
$\mathcal{X}_{\text{tier}}$	20; $[-64, 64]$	fixed near-input schedule and clamping interval
q_{far}	3, plus 2 ordered multi-parameter probes	deterministic far probes with magnitude at most 512
Δ_{rel}	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	local relational perturbations
h_{dense}	3	successor window used to establish a relation base
b_{rel}	96	cap on relation bases stratified across positives
h_{exit}	24	continuation horizon after a witnessed exit
$b_{\text{rel}}^-, b_{\text{exit}}^-$	48, 12	relational and post-exit negative-trace budgets
u_{max}	2×10^6	execution-iteration safety cap
w_c, w_g	1.0, 0.3	coverage and group-credit weights
m_{max}	20 clauses	cap applied independently to each response
$\mathcal{K}$	$\{1, 4, 8, 16, 32\}$	reported local-model response budgets
n_{probe}	128; 100; 32	saved responses per task (8B; A3B/Llama; remaining base models)
ϑ	1.0	decoding temperature except for Loopy (Table 25)
p	1.0	nucleus-sampling probability
L_{api}	8,192 tokens	maximum API-model completion length
L_{local}	2,048 tokens	maximum local-model completion length
g_{sft}	16 responses	group sampled per SFT strengthening round
t_{sft}	5 rounds	maximum per loop; at most 80 responses
τ	0.5	SFT acceptance threshold on composed negative coverage
h_{max}	500	maximum Houdini verification rounds
$\mathcal{P}_{\text{verify}}$	Alt-Ergo and Z3	prover portfolio; Typed memory model
δ	5 seconds per goal	per-prover goal budget used by LoopVerify
$\delta_{\text{short}}, \delta_{\text{long}}$	300, 900 seconds	verification caps in the pre-/post-filtering comparison

Table 3: Parameter values for execution sampling (§4.2), SFT synthesis (§4.3), reward computation (§4.4), inference, and verification and evaluation.

The fixed near-input tier order is $\langle 0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9 \rangle$.

C.1 LOOP-VERIFIER REALIZATION

The implementation generates establishment and preservation goals for each clause in the ordered current family $D = \langle d_1, \dots, d_m \rangle$. Establishment of d_k may use earlier annotations d_j , $j < k$, as hypotheses. Preservation uses the full current family as loop-head hypotheses and may use earlier annotations in the successor state. These hypotheses are available during the current round regardless of the status of their own goals. The verifier simplifies and splits the obligations by control-flow path. A clause receives valid only when every generated establishment and preservation subgoal is proved; timeout records an exhausted prover budget, and all other outcomes map to unknown. Houdini reruns the survivors after each deletion round.

C.2 NEGATIVE-TRACE SAMPLING

Algorithm 5 specifies the structured sampler’s selection order. The routines below define its execution, proposal, cleaning, and budget rules. These candidate negatives provide target-independent behavioral probes. Generation and cleaning depend only on visible program information.

Algorithm 5 Target-hidden candidate-negative sampling

Input: masked loop $\mathcal{L}$, input schedule U , configuration Θ
Output: observed states $\mathcal{E}$, candidate traces $\mathcal{N}$

- 1: $\tilde{\mathcal{L}} \leftarrow \text{Determinize}(\mathcal{L}) \triangleright \text{execution only}$
- 2: $(T, O, a) \leftarrow \text{Trace}(\tilde{\mathcal{L}}, U, \Theta) \triangleright a: \text{any run capped}$
- 3: $\mathcal{E} \leftarrow \text{UniqueHeads}(T); Z \leftarrow \emptyset; \mathcal{N} \leftarrow \langle \rangle$
- 4: $M \leftarrow \text{Movable}(\mathcal{L}); Q \leftarrow \text{StratifiedBases}(\mathcal{E}, 96)$
- 5: $b \leftarrow \text{PerturbationBlocked}(\mathcal{L})$
- 6: **if** $\neg b$ **then**
- 7: $C \leftarrow \text{RelationProposals}(Q, T, M, a)$
- 8: $C \leftarrow \text{Clean}(C, \mathcal{E}, Z)$
- 9: $R \leftarrow \text{RoundRobinNearest}(C, 48)$
- 10: append $\langle s \rangle$ for each $s \in R$ to $\mathcal{N}$; add their keys to Z
- 11: $e \leftarrow 0$
- 12: **for each** run u in increasing order with $e < 12$ **do**
- 13: $\nu \leftarrow \text{Clean}(O_u, \mathcal{E}, Z)$
- 14: **if** $\nu \neq \emptyset$ **then** append ν to $\mathcal{N}$, add its keys to Z , and increment e
- 15: **if** $\neg b$ **then** $\mathcal{N} \leftarrow \mathcal{N} \parallel \text{RandomFill}(Q, M, 60 - |\mathcal{N}|, \mathcal{E}, Z)$
- 16: **return** $(\mathcal{E}, \mathcal{N})$

Execution and state identity. Determinize replaces each supported zero-argument oracle call site with a fresh sampled integer parameter, fixed throughout a run. This explores a restricted set of oracle behaviors; the original program is still used for generation and proof. A state key includes its current valuation and both function-entry and loop-entry snapshots. Perturbations preserve these snapshots, so states from different contexts are not conflated. The requested run count is 12, doubled for oracle-driven bodies; the input schedule and execution cap are in Table 3. T records entry, sampled heads, and exits with run/iteration coordinates. After an uncapped exit, the instrumented executor replays the body for at most 24 steps without the original guard to form O_u . Bodies containing `return`, `goto`, or labels disable this replay. Capped runs contribute no post-exit states; detected abnormal runs are skipped. If no observed states remain, sampling reports failure.

Perturbation eligibility and bases. M contains tracked variables assigned by the body. If none are found, the implementation falls back to tracked non-parameter variables, or all tracked variables if that set is also empty. Untracked persistent assignments, unsupported body calls, or unsupported local pointer/array declarations block both relational and random perturbations, but do not by themselves discard executed post-exit traces. Ordinary block-local scalar temporaries are not treated as persistent state. Q allocates at most 96 bases round-robin across runs and entry contexts, keeping dense prefixes, endpoints, and evenly spaced positions where quotas permit.

Relation proposals. A base must have the next three head positions recorded, or be an observed terminal with a guard-true predecessor and guard-false current state. Terminal bases are added even if omitted from Q , but are disabled when any run was capped. For each base, perturb one axis by d , or a pair by (d, d) or $(d, -d)$. Interior offsets are $d \in \pm\{1, 2\}$; terminal offsets are $d \in \pm\{1, 2, 3, 5, 8\}$. Retain a proposal only if every coordinate stays within its context's sampled minimum/maximum envelope and the guard has the same known truth value. Reject changes that are an integer multiple of the observed forward step (or the predecessor step if no successor is recorded). This tangent test avoids obvious movements along witnessed dynamics. It provides a local screen for reachability. Envelope checks prevent simple sampled marginal bounds from separating this family, but do not constrain post-exit or random candidates.

Cleaning and its limits. Clean removes keys in $\mathcal{E}$, previously selected keys Z , and duplicates within its input. It also drops a state if the entry screen succeeds: parameter values agree with their function-entry values, every retained local-initialization equality evaluates to true, and the precondition does not evaluate to false. An unknown initialization equality makes this screen fail; an unknown precondition does not. Consequently the screen provides a conservative initialization check. For a post-exit group, cleaning removes individual states while preserving the order of the remainder; an empty group is discarded.

Exclusion from $\mathcal{E}$, failure of the entry screen, and departure from a local tangent do not prove unreachability. Likewise, overriding an exit creates a counterfactual execution, but its states may occur on another legal execution. No general unreachability solver is called. Under sound inductive verification and consistent state-evaluation semantics, a truly reachable state cannot be rejected by a valid invariant. Such contamination can nevertheless dilute coverage and affect relative rewards or SFT acceptance. The final proof guarantee is supplied by the verifier, not by the sampler’s labels.

Selection, random fill, and scoring units. Relation candidates are bucketed by interior/terminal status, changed axes, and change signs. Stable round-robin selection takes up to 48 candidates, preferring smaller ℓ_1 changes within each bucket. Only selected keys are committed to Z . Next, up to 12 nonempty post-exit groups are selected in run order. Thus cross-family deduplication gives priority to relation, then post-exit, then random fill.

RandomFill draws a base uniformly from Q , chooses one or two axes uniformly (one if only one is available), and samples a nonzero integer offset independently per axis. It first uses magnitudes 1–34, then 35–128 if needed, with seed `seedxor0x4C4F4F50`. Each stage permits at most $\max(1024, 100q)$ attempts for the requested remaining capacity q . Candidates use the same observed-state, entry, and duplicate checks, but not the relation-specific envelope, guard, or tangent constraints. Accepted states immediately enter Z and each forms a singleton trace. The random baseline uses this routine alone with $q = 60$. Neither mode is guaranteed to fill all 60 slots when admissible candidates are scarce.

Every selected singleton or cleaned continuation contributes one coverage unit, rejected if any retained clause is definitely false at any witness. This avoids counting a long continuation repeatedly, although longer traces still offer more rejection opportunities. Unknown evaluations do not reject a trace. Empty-negative handling is specified in Algorithm 4 and Section 4.3.

C.3 CLAUSE NORMALIZATION AND OPTIMIZATION SETTINGS

Appendix A.7 defines the group-normalized advantage, token-level importance ratio, sampled conditional KL term, and clipped GRPO objective. Clause deduplication, which forms the clause set scored above, normalizes comparison direction, equality sides, parentheses, immediate commutative operands, harmless identities, and Boolean association, but avoids transformations that require additional arithmetic assumptions. Repeated clauses therefore collapse without affecting coverage, and a unique zero-coverage clause costs nothing because it may support another clause or the hidden target. Coverage and Shapley credit each lie in $[0, 1]$; clauses beyond the admission cap incur no reward penalty. Empty-negative groups use the binary fallback in Algorithm 4.

D GENERATION PROMPT

The canonical evaluation template appears below; every target is stripped before insertion, and general annotation rules are supplied by the common system prompt. Both the SFT and RL data use this single prompt pair.

Generation prompt

You are given a C function. Produce the strongest inductive ACSL loop invariants that capture the loop’s behavior (conservation/relational laws + tight progress bounds). Output ONLY invariant lines, each formatted exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}

The accompanying system prompt appears verbatim below.

1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457

System prompt

```
You are a formal verification expert specializing in ACSL loop invariant
synthesis for Frama-C/WP.

## LOOP INVARIANT DEFINITION

A loop invariant is a property that:
1. Holds before the first loop iteration (initialization).
2. Is preserved by every loop iteration when the loop guard is true
   (inductiveness).
3. Combined with loop exit ('!guard'), helps establish required
   loop-exit facts.

## RULES

- Add one core conservation/relational equality that explains loop
  updates (state transition law).
- Add minimal progress bounds for loop counters and key arithmetic
  variables.
- Prefer strong invariants; avoid weak/tautological ones.
- Do not add unrelated invariants just to increase count.
- MUST NOT modify or rewrite any 'unknown()'/ 'unknownN()' call.
- '\at(v, Pre)' can ONLY be used with function parameters, never with
  local variables initialized inside the function.
- '\at(v, LoopEntry)' denotes the value of v at the start of the loop
  and can ONLY be used with local variables initialized before the
  loop (using v = unknown();).
- Loop guard is NOT an invariant; weaken strict guards to non-strict
  bounds at exit.
- Parenthesize mixed equality and relational expressions so their
  grouping is explicit.
- Use 'loop invariant' only as a line prefix, never inside expressions.
- Emit scalar invariant expressions only; do not declare predicates,
  logic functions, axioms, or lemmas.
- Do NOT use the ternary '? :' operator; split cases with '==>' instead.
- '' is bitwise XOR in C/ACSL, not exponentiation --- write 'x*x*x'
  for x cubed.
- Do NOT place a C boolean directly in arithmetic ('x + (a > b)' is
  invalid); use '==>' to split cases.
- Only use variables declared in the given function.
- Operators: comparison '==' '!=' '<' '<=' '>' '>='; logical '&&' '||'
  '!' '==>' '<=>'; arithmetic '+' '-' '*' '/' '%'.

## ACSL INVARIANT SYNTAX

Allowed expression forms (all verified by Frama-C/WP):

1. **Bounds**: '0 <= i', 'i <= n', 'x >= 1'
2. **Linear equality**: 'i + c == n', '2 * s == i * (i - 1)',
  'x == q * y + r'
3. **Polynomial equality**: 'x == n * n * n',
  '6 * s == i * (i + 1) * (2 * i + 1)'
4. **Relational identity** (multi-var algebraic law):
  '1 + x * z - x - z * y == 0', 'p * s - r * q == 1'
5. **Implication**: 'i > 0 ==> s >= i', 'a != 0 ==> q >= 1'
6. **Modular**: 'i % 5 == 0', 'r >= 0 && r < y'
7. **Bitshift** (power-of-2 only): 'p == 1 << i', 's == (1 << i) - 1'
8. **Conjunction**: 'c >= 1 && c <= k'
9. **\at for params**: 'n == \at(n, Pre)' (function parameters only)

Forbidden:
- '' for exponentiation (it is XOR) --- use repeated '*' or '<<' for
  powers of 2.
- '\product', '\sum', '\factorial' --- not ACSL scalar operators.
- '? :' ternary --- rewrite with '==>'.
- 'old(x)' --- use '\at(x, Pre)'.
- Type casts '(int)', '(long)' --- unnecessary in ACSL logic.
- Function calls, predicates, lemmas, axioms --- emit only scalar
  expressions.

Operators (complete list):
comparison: '==' '!=' '<' '<=' '>' '>='
logical: '&&' '||' '!' '==>' '<=>'
arithmetic: '+' '-' '*' '/' '%' '<<' '>>'

## OUTPUT

Return ONLY invariant lines, one per line, formatted exactly as:
'loop invariant <property>;'
Do not return code fences, 'loop assigns', the surrounding C function,
or explanations.
```

E TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and its prover backend. It accepts program source, an array of model responses, one of the

four reward variants defined below, and either random or structured negative sampling. The reward and sampler choices are independent. The response reports each rollout’s reward together with its coverage. Responses may be raw LLM text, invariant lists, or annotated code.

F DO TRAINING AND EVALUATION SHARE PROGRAMS OR ONLY STRUCTURAL PATTERNS?

We compare the current RL (9,024 records and 9,024 distinct program sources) and SFT (33,346 records and 31,865 distinct program sources) training sets against all 832 evaluation programs after target removal. We canonicalize simple break-based loop guards and apply three increasingly aggressive normalizations: exact tokens, alpha-renamed identifiers, and alpha-renaming with non-trivial integer constants abstracted. No training program matches an evaluation program under these representations (Table 4).

We separately measure coarse structural overlap. A structural cell combines the control-flow profile, guard kind, nonlinearity, nondeterminism, and a variable-count band. The evaluation set occupies 122 such cells. RL and SFT share 38 and 41 of them, respectively; their union shares 41 cells. Equivalently, 524 (63.0%), 529 (63.6%), and 529 (63.6%) of the 832 evaluation programs fall in a cell represented by RL, SFT, and their union. The record-weighted total-variation distances over cells are 0.688 for RL, 0.651 for SFT, and 0.633 for their union, so the training and evaluation distributions remain different. Table 5 gives interpretable feature marginals behind this joint-cell comparison. All statistics in this section are recomputed from the current training files.

	RL	SFT	RL $\cup$ SFT
Records	9,024	33,346	42,370
Distinct program sources	9,024	31,865	33,479
Exact-token matches	0	0	0
Alpha-renamed matches	0	0	0
Alpha-renamed, constant-abstracted matches	0	0	0
Shared evaluation cells	38 / 122	41 / 122	41 / 122
Evaluation programs in shared cells	524 (63.0%)	529 (63.6%)	529 (63.6%)
Training records in evaluation-supported cells	5,056 (56.0%)	27,275 (81.8%)	32,331 (76.3%)
TV distance over structural cells	0.688	0.651	0.633

Table 4: Instance and structural-distribution audit of the current training sets against the 832 target-removed evaluation programs. Distribution statistics count training records, matching their sampling frequency.

Feature (% of records/programs)	Evaluation	RL	SFT
Constant guard	24.6	10.3	4.6
Single-variable guard	31.4	38.8	44.0
Variable-vs.-variable guard	37.3	49.4	50.6
Compound guard	6.7	1.5	0.8
Nonlinear	2.5	14.3	10.8
Nondeterministic	28.6	41.5	14.9
≤ 2 variables	35.1	14.6	46.2
3–4 variables	33.9	23.1	12.0
≥ 5 variables	31.0	62.3	41.8

Table 5: Marginal structural distributions of evaluation programs and current training records. The structural-cell TV distances in Table 4 use the joint feature distribution.

G TRAINING-DATA CURATION

Program sources. The raw training-program pool is drawn primarily from SV-COMP and SyGuS (Beyer, 2023; Alur et al., 2013). We retain scalar-integer single-loop programs and hide target

assertions and postconditions in the model-facing input. The RL selection below operates on this program pool; SFT-target construction generates verified invariant annotations for the training programs.

The RL and SFT training sets use the target-hidden scalar-integer single-loop interface. The reported RL experiments and the released RL dataset use the same 9,024 records. SFT targets are inductive invariant sets synthesized by the multi-round procedure in Section 4.3 and accepted by the loop verifier. The only learned generator is the open-weight model being trained; target acceptance and RL reward computation use program execution and verification. Table 6 summarizes the current training sets.

Training set	Size and composition
RL set	9,024 records and distinct program sources; 1,794 loop shapes
SFT set	33,346 records over 31,865 distinct program sources
RL-SFT exact-source overlap	7,410 distinct program sources

Table 6: Current training sets used in the experiments. RL loop shapes abstract identifiers and nontrivial constants in the loop guard and body; source overlap compares the program text in training prompts.

RL selection. The RL set contains unchanged rows selected from the raw program pool, with no repeated source text. Whole-source family caps admit at most three records per identifier-normalized family and eight per family after also abstracting nontrivial constants. The selected set spans 6,760 and 4,196 families under these two representations, respectively. These whole-source families differ from the loop-only shapes in Table 6. Static complexity criteria yield 529 easy, 7,745 medium, and 750 challenging programs; these labels summarize static program complexity. Selection rejects unsupported state, abnormal executions, insufficient negative traces, and sampler runs exceeding 15 seconds. All selected programs produced 60 negative-trace groups under the audited sampler with 12 runs and seed 0. Separate training diagnostics assess within-group reward variance. The final records are shuffled with seed 20260906 and aligned to 141 batches of 64.

H EXPERIMENTAL PROTOCOL DETAILS

The evaluation workload composition is given in Section 5.1; the 466 Loopy programs are the integer subset of its original 469-program corpus (three floating-point programs excluded), and unsupported inputs remain failures in the common denominator.

Before model invocation, we remove assert and ensures predicates, executable assertion calls, and conventional error-label names; preconditions and executable semantics remain visible. CRAFT evaluation runs use the canonical generation template, extractor, canonical-key deduplication, and Houdini implementation; external tools retain their own orchestration. Each model retains its serving interface and chat template. All API and locally served CRAFT models use reasoning-disabled decoding. Decoding uses temperature ϑ , nucleus parameter p , and completion caps L_{api} and L_{local} for API and local models, respectively, without an additional top- k , repetition penalty, re-roll, or target feedback. Each probe curve reuses the same saved response pool per task. The model-specific pool sizes and the common reported budget grid are listed in Table 3; every pool contains at least k_{max} responses. For task t , let n_t be its saved pool size, $s_{t,i} \in \{0, 1\}$ the standalone verifier verdict, and $c_t = \sum_{i=1}^{n_t} s_{t,i}$. Let $v_{t,k} \in \{0, 1\}$ indicate whether filtering and composing the first k saved responses succeeds. The archived evaluations report

$$\widehat{\text{pass@}k} = \frac{100}{N} \sum_{t=1}^N \left(1 - \frac{\binom{n_t - c_t}{k}}{\binom{n_t}{k}} \right),$$

$$\text{compose@}k = \frac{100}{N} \sum_{t=1}^N v_{t,k},$$

with $\binom{n_t - c_t}{k} = 0$ when $n_t - c_t < k$. The archived pass estimator averages over response subsets; compose evaluates a saved prefix.

Integer-count projection of reported pass values. For a reported pass percentage p on N programs, the displayed value is projected onto the nearest integer-count grid:

$$\tilde{c} = \left\lfloor \frac{Np}{100} + \frac{1}{2} \right\rfloor, \quad \tilde{p} = \frac{100\tilde{c}}{N}.$$

Tables and plots display $\tilde{p}$ to two decimal places, retaining the pass@ k label. On the 50-program NLA stratum, values therefore advance in 2-point increments. The projection is applied separately to each reported population; projected stratum counts need not sum to the independently projected whole-workload count.

These are rounded reported estimates, not recomputed per-program success counts. Original values and projected counts are preserved in the data audit. Projection does not recover first-prefix verdicts, establish paired sampling with compose, or remove sampling uncertainty. Ratios and pass differences in the text use the displayed projected rates. Compose values retain their reported evaluations.

Training configuration. Tables 7 and 8 record the supplied SFT and RL run configurations. SFT uses 3 epochs; RL uses 2 epochs on 9,024 programs. Reward and sampler constants are reported in Table 3.

Field	Value
Learning rate η_{sft}	1×10^{-5}
Epochs e_{sft}	3
Batch (per device $\times$ accumulation $\times$ GPUs)	global batch 32; $2 \times 2 \times 8$ or $1 \times 4 \times 8$
Sequence cutoff L_{sft}	2,048
DeepSpeed	ZeRO-3
Scheduler	cosine

Table 7: SFT training configurations.

Field	Value
Batch size b_{rl}	64
Learning rate η_{rl}	1×10^{-6}
KL coefficient β	0.001
Advantage stabilizer δ_{adv}	10^{-6}
Rollouts per prompt g_{rl}	8
Tensor parallelism p_{tp}	1–2
GPU memory utilization μ_{gpu}	0.3–0.5
Epochs e_{rl}	2

Table 8: RL training configurations.

I WHY THINKING IS DISABLED FOR LOCAL MODELS

The reasoning-disabled API experiments use compatible endpoint controls; Claude Sonnet disables adaptive thinking with a zero thinking budget. Local models use reasoning-disabled decoding during synthetic-data construction, training rollouts, and evaluation.

Explicit chain-of-thought adds reasoning tokens to every rollout, with total generation scaling with the sampling budget. The required output is a short, machine-parseable set of ACSL clauses. These clauses admit clause-level filtering and cross-rollout composition: Houdini can remove a failing clause, retain its siblings, and merge survivors from different rollouts. Reasoning traces are generated and consumed as whole sequences and are not included in this clause-level composition operator.

J COMPLETE RESULTS FOR MODEL TRAINING CONFIGURATIONS

Model naming. Local models are named in training order as *Backbone + SFT + RL*, omitting a stage when it was not applied. SFT defaults to training on targets constructed by composing clauses across responses. SFT (single) explicitly identifies targets constructed by filtering individual responses without cross-response composition. The four RL rewards are Binary, Whole-rollout, Clause-decomposed, and Full. RL defaults to Full; other rewards are named explicitly. Full adds Shapley credit with weight 0.3 to Clause-decomposed. For example, Qwen3-8B + SFT + RL uses Full, while Qwen3-8B + SFT + RL (Clause-decomposed) names the corresponding reward ablation. Qwen3-4B + SFT (single) uses single-response-data SFT alone, and Qwen3-8B + RL (Binary) denotes Binary-reward RL directly from Bare. These suffixes specify training; $\text{pass}@k$ and $\text{compose}@k$ remain the evaluation metrics. Reward definitions are given in Appendix K.3.

Data and layout. This appendix records full pass/compose results for the model training configurations: six Bare reference models; four SFT models; four SFT (single) models; four models trained by RL directly from Bare; four trained by RL after SFT; and the Qwen3-8B reward variants from each initialization. Additional tables retain cross-backbone Whole-rollout controls. Each table reports all available budgets and the Linear, NLA, Loopy, and All strata. Experimental comparisons, figures, and interpretation are collected separately in Appendix K.

Pass values use the disclosed nearest-integer-count projection in Appendix H; original reported values remain preserved in the audit. The main Qwen3-8B + SFT + RL evaluation uses Full and reaches 69.23% $\text{compose}@1$; the same Full evaluation is used throughout the results and reward comparisons. Clause-decomposed reaches 67.91%. Other configurations use their latest available results; the Llama Whole-rollout rows use the newly supplied complete evaluation. A dash indicates an unavailable result.

J.1 BARE MODELS

Table 9: Complete results for the six Bare models. Each cell is a percentage; All / 832 is the whole-workload result. Bold marks the highest reported value in each metric column across all models and budgets in this table, including ties.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-1.7B	1	5.38	19.30	2.00	6.00	6.22	21.03	5.77	19.47
	4	9.81	29.75	4.00	8.00	12.23	30.69	10.82	28.97
	8	12.34	33.23	6.00	8.00	15.45	35.62	13.70	33.05
	16	15.19	35.76	8.00	10.00	18.88	37.77	16.71	35.34
	32	18.35	38.29	8.00	14.00	22.10	40.13	19.83	37.86
Qwen3-4B	1	5.06	33.54	0.00	8.00	6.01	35.19	5.29	32.93
	4	10.44	44.94	2.00	10.00	10.52	47.21	9.98	44.11
	8	13.29	50.00	2.00	10.00	13.09	50.00	12.50	47.60
	16	15.82	52.53	4.00	12.00	15.88	53.65	15.14	50.72
	32	18.35	54.11	6.00	12.00	18.88	56.22	17.91	52.76
Qwen3-8B	1	6.01	37.97	2.00	6.00	7.08	41.20	6.43	37.86
	4	12.97	50.95	4.00	6.00	14.16	55.15	13.24	50.60
	8	17.41	54.75	6.00	10.00	18.45	58.58	17.33	54.21
	16	21.84	56.65	6.00	10.00	23.18	61.16	21.62	56.37
	32	25.63	58.23	6.00	10.00	28.33	62.66	25.96	57.81
Qwen3-14B	1	6.01	48.10	2.00	4.00	9.44	44.64	7.69	43.51
	4	11.71	61.39	2.00	10.00	17.17	61.37	14.18	58.29
	8	14.87	64.56	4.00	12.00	21.46	64.81	18.03	61.54
	16	18.67	66.14	4.00	14.00	26.18	67.38	22.00	63.70
	32	22.78	66.46	6.00	14.00	30.47	68.67	26.08	64.54
Qwen3-30B-A3B	1	1.27	16.14	0.00	0.00	1.50	20.17	1.44	17.43
	4	4.75	29.43	0.00	4.00	4.94	34.33	4.57	30.65
	8	8.23	36.71	0.00	4.00	7.94	39.91	7.69	36.54
	16	13.29	43.35	0.00	8.00	12.66	45.28	12.14	42.31
	32	18.67	43.99	2.00	8.00	18.45	47.64	17.55	43.87
Llama 3.1-8B	1	0.63	28.16	0.00	6.00	0.64	30.04	0.60	27.88
	4	2.22	48.73	0.00	12.00	2.36	47.85	2.16	46.03
	8	4.11	54.43	0.00	12.00	4.08	53.22	3.85	51.20
	16	6.96	55.06	0.00	12.00	6.44	53.65	6.25	51.68
	32	10.76	55.06	0.00	12.00	10.09	54.51	9.74	52.16

J.2 SFT MODELS

Table 10: Complete results for four SFT models using composed training targets. Each cell is a percentage; All / 832 is the whole-workload result. Bold marks the highest reported value in each metric column across all models and budgets in this table, including ties.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-4B + SFT	1	43.67	65.82	2.00	8.00	42.27	66.31	40.38	62.62
	4	58.54	78.48	4.00	10.00	58.37	76.82	55.29	73.44
	8	63.29	81.01	6.00	12.00	63.52	79.83	59.98	76.20
	16	67.41	82.28	8.00	12.00	67.81	80.90	64.06	77.28
	32	71.20	82.28	10.00	12.00	71.46	81.97	67.79	77.88
Qwen3-8B + SFT	1	45.57	68.40	4.00	8.00	43.56	67.00	41.93	63.90
	4	61.08	77.20	6.00	10.00	59.44	78.30	56.82	73.80
	8	65.51	80.10	8.00	10.00	63.95	80.70	61.11	76.20
	16	68.99	81.30	8.00	12.00	67.81	81.50	64.68	77.30
	32	72.47	81.60	10.00	12.00	71.24	82.40	67.98	77.90
Qwen3-14B + SFT	1	50.32	70.25	4.00	12.00	47.21	69.53	45.67	66.35
	4	63.92	81.33	8.00	16.00	61.37	80.04	59.13	76.68
	8	68.04	82.28	10.00	16.00	66.31	82.40	63.46	78.37
	16	71.52	82.91	12.00	16.00	70.60	83.48	67.31	79.21
	32	74.05	82.91	14.00	16.00	74.25	83.69	70.55	79.33
Llama 3.1-8B + SFT	1	43.67	67.09	2.00	8.00	40.34	63.09	39.18	61.30
	4	56.01	75.32	4.00	14.00	55.36	72.96	52.64	70.31
	8	60.13	77.53	6.00	14.00	60.09	76.39	56.85	73.08
	16	63.61	78.48	8.00	14.00	63.95	77.47	60.46	74.04
	32	66.77	78.80	10.00	14.00	67.38	77.90	63.70	74.40

J.3 SFT MODELS WITH SINGLE-RESPONSE TARGETS

Table 11: Complete results for four SFT (single) models, September 10, 2026. Each cell is a percentage; All / 832 is the whole-workload result. Bold marks the highest reported value in each metric column across all models and budgets in this table, including ties.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-4B + SFT (single)	1	20.89	28.48	0.00	0.00	17.81	22.32	17.91	23.32
	4	34.18	40.82	0.00	0.00	28.11	32.62	28.73	33.77
	8	40.51	45.57	2.00	2.00	33.91	39.70	34.50	39.66
	16	43.67	49.05	4.00	6.00	38.63	43.99	38.46	43.63
	32	47.47	52.85	4.00	10.00	44.21	50.86	43.03	49.16
Qwen3-8B + SFT (single)	1	21.84	27.22	0.00	0.00	20.82	22.96	19.95	23.20
	4	33.23	38.29	0.00	0.00	30.04	32.19	29.45	32.57
	8	38.92	43.04	2.00	2.00	35.41	38.41	34.74	37.98
	16	42.72	48.10	2.00	2.00	39.70	42.49	38.58	42.19
	32	46.20	51.90	8.00	8.00	45.06	49.14	43.27	47.72
Qwen3-14B + SFT (single)	1	21.20	27.53	0.00	0.00	18.24	20.60	18.27	22.00
	4	31.01	36.08	0.00	0.00	29.83	32.40	28.49	31.85
	8	36.39	39.87	0.00	2.00	34.76	38.20	33.29	36.66
	16	42.41	46.20	0.00	2.00	41.20	44.85	39.18	42.79
	32	45.89	50.00	2.00	6.00	46.57	50.21	43.63	47.48
Llama 3.1-8B + SFT (single)	1	19.94	25.95	0.00	0.00	18.67	21.24	18.03	21.75
	4	30.38	36.39	0.00	0.00	27.47	30.04	26.92	30.65
	8	35.13	38.61	0.00	2.00	31.76	34.55	31.13	34.13
	16	38.92	42.09	0.00	2.00	35.62	40.13	34.74	38.58
	32	40.82	46.52	0.00	4.00	39.27	44.21	37.50	42.67

J.4 RL DIRECTLY FROM BARE

Table 12: Latest available results for four models trained by RL directly from Bare, using the default reward. Each cell is a percentage; All / 832 is the whole-workload result. Bold marks the highest reported value in each metric column across all models and budgets in this table, including ties.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-4B + RL	1	2.22	52.20	0.00	10.00	1.50	50.60	1.68	48.80
	4	4.11	63.60	0.00	10.00	3.22	61.60	3.37	59.30
	8	5.38	66.50	0.00	12.00	4.29	65.20	4.45	62.50
	16	6.65	69.30	2.00	12.00	5.58	67.20	5.65	64.70
	32	8.23	70.30	2.00	12.00	7.30	68.70	7.33	65.90
Qwen3-8B + RL	1	2.53	54.43	0.00	6.00	3.43	54.94	2.88	51.80
	4	4.75	61.71	0.00	10.00	6.44	65.02	5.41	60.46
	8	7.59	66.46	0.00	12.00	8.80	68.24	7.81	64.18
	16	9.18	71.52	0.00	12.00	10.94	73.18	9.62	68.87
	32	10.44	73.10	0.00	12.00	12.66	75.32	11.06	70.67
Qwen3-14B + RL	1	9.49	66.80	0.00	14.00	12.02	70.00	10.46	65.40
	4	14.87	72.80	2.00	14.00	18.03	74.70	15.75	70.30
	8	18.04	74.40	2.00	14.00	20.82	77.00	18.75	72.20
	16	21.20	75.60	4.00	14.00	23.61	78.50	21.51	73.60
	32	24.68	76.60	4.00	14.00	25.97	79.40	24.16	74.40
Llama 3.1-8B + RL	1	6.01	51.90	0.00	6.00	3.86	46.40	4.45	46.00
	4	10.76	58.90	0.00	8.00	7.94	55.20	8.65	53.70
	8	12.66	60.40	0.00	8.00	10.09	58.60	10.46	56.20
	16	14.87	62.30	0.00	8.00	11.80	59.70	12.26	57.60
	32	16.77	63.30	0.00	10.00	13.52	61.20	13.94	58.90

J.5 RL AFTER SFT

Table 13: Latest available results for four models trained by RL after SFT. All four models use the default Full reward. Each cell is a percentage; All / 832 is the whole-workload result. Bold marks the highest reported value in each metric column across all models and budgets in this table, including ties.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-4B + SFT + RL	1	36.39	72.78	2.00	12.00	37.77	69.96	35.10	67.55
	4	51.27	79.11	4.00	14.00	48.93	79.83	47.12	75.60
	8	57.91	81.33	6.00	16.00	55.15	82.40	53.25	78.00
	16	61.71	81.96	6.00	16.00	61.37	83.48	58.17	78.85
	32	66.77	82.28	8.00	16.00	65.88	83.91	62.74	79.21
Qwen3-8B + SFT + RL	1	32.91	75.00	2.00	14.00	32.40	71.24	30.78	69.23
	4	46.52	79.75	4.00	14.00	45.71	78.76	43.44	75.24
	8	50.95	80.70	4.00	14.00	49.79	80.90	47.47	76.80
	16	54.75	80.70	4.00	14.00	53.00	82.40	50.63	77.64
	32	57.91	81.33	4.00	14.00	55.79	83.26	53.47	78.37
Qwen3-14B + SFT + RL	1	40.51	72.78	2.00	12.00	36.27	73.18	35.82	69.35
	4	53.16	77.85	8.00	14.00	50.21	79.83	48.80	75.12
	8	57.91	79.75	8.00	14.00	54.51	80.90	53.00	76.44
	16	60.44	81.01	8.00	14.00	57.30	82.19	55.53	77.64
	32	62.34	81.01	8.00	14.00	60.52	83.26	58.05	78.25
Llama 3.1-8B + SFT + RL	1	39.87	66.77	4.00	8.00	36.27	64.38	35.70	61.90
	4	52.53	75.63	4.00	12.00	50.21	75.97	48.32	72.00
	8	57.59	78.16	6.00	12.00	54.94	78.54	53.00	74.40
	16	60.76	79.75	6.00	12.00	59.87	78.97	56.97	75.24
	32	63.92	79.75	6.00	12.00	62.88	79.83	59.86	75.72

J.6 REWARD VARIANTS FROM BARE

Table 14: Qwen3-8B reward variants trained from Bare, September 10, 2026. Each cell is a percentage; All / 832 is the whole-workload result. Bold marks the highest reported value in each metric column across all models and budgets in this table, including ties.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-8B + RL (Binary)	1	4.75	0.00	0.00	0.00	4.72	0.00	4.45	4.55
	4	4.75	0.00	0.00	0.00	4.72	0.00	4.45	4.55
	8	4.75	0.00	0.00	0.00	4.72	0.00	4.45	4.55
	16	4.75	0.00	0.00	0.00	4.72	0.00	4.45	4.55
	32	4.75	0.00	0.00	0.00	4.72	0.00	4.45	4.55
Qwen3-8B + RL (Whole-rollout)	1	17.41	24.68	2.00	4.00	15.67	19.96	15.50	20.79
	4	18.67	26.27	2.00	4.00	16.74	20.82	16.59	21.88
	8	18.67	26.58	2.00	4.00	17.17	21.24	16.83	22.24
	16	19.30	26.58	2.00	4.00	18.24	22.32	17.67	22.84
	32	19.62	26.90	2.00	4.00	19.31	22.96	18.39	23.32
Qwen3-8B + RL (Clause-decomposed)	1	9.18	52.53	2.00	6.00	9.66	50.86	9.01	48.80
	4	17.09	61.39	2.00	8.00	16.52	60.52	15.87	57.69
	8	20.89	63.92	2.00	12.00	19.31	63.73	18.87	60.70
	16	22.78	66.77	2.00	12.00	20.60	67.17	20.31	63.70
	32	24.37	69.30	2.00	14.00	21.89	69.96	21.63	66.35
Qwen3-8B + RL	1	2.53	54.43	0.00	6.00	3.43	54.94	2.88	51.80
	4	4.75	61.71	0.00	10.00	6.44	65.02	5.41	60.46
	8	7.59	66.46	0.00	12.00	8.80	68.24	7.81	64.18
	16	9.18	71.52	0.00	12.00	10.94	73.18	9.62	68.87
	32	10.44	73.10	0.00	12.00	12.66	75.32	11.06	70.67

J.7 REWARD VARIANTS AFTER SFT

Table 15: Qwen3-8B reward variants trained after SFT. Full uses the main evaluation; the other rewards use the reward-ablation report. Each cell is a percentage; All / 832 is the whole-workload result. Bold marks the highest reported value in each metric column across all models and budgets in this table, including ties.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-8B + SFT + RL (Binary)	1	8.54	7.59	0.00	0.00	9.87	7.51	8.77	7.09
	4	11.08	10.76	0.00	0.00	12.66	12.02	11.30	10.82
	8	13.29	13.61	0.00	0.00	14.59	14.59	13.22	13.34
	16	15.51	15.82	0.00	0.00	15.02	15.24	14.30	14.54
	32	17.72	18.04	0.00	0.00	15.67	16.31	15.50	15.99
Qwen3-8B + SFT + RL (Whole-rollout)	1	41.77	46.20	4.00	6.00	40.56	45.92	38.82	43.63
	4	48.73	53.16	6.00	8.00	47.00	52.58	45.19	50.12
	8	51.58	56.01	8.00	8.00	51.07	56.44	48.68	53.37
	16	53.80	57.28	8.00	8.00	53.43	60.30	50.84	56.01
	32	57.28	59.81	8.00	8.00	56.22	63.09	53.73	58.53
Qwen3-8B + SFT + RL (Clause-decomposed)	1	36.39	71.84	2.00	12.00	39.27	71.24	35.94	67.91
	4	51.90	80.06	8.00	12.00	55.79	79.83	51.44	75.84
	8	56.01	80.70	8.00	12.00	58.15	81.12	54.33	76.80
	16	60.13	81.01	8.00	12.00	61.37	82.40	57.69	77.64
	32	63.92	81.01	8.00	12.00	65.24	83.48	61.30	78.25
Qwen3-8B + SFT + RL	1	32.91	75.00	2.00	14.00	32.40	71.24	30.78	69.23
	4	46.52	79.75	4.00	14.00	45.71	78.76	43.44	75.24
	8	50.95	80.70	4.00	14.00	49.79	80.90	47.47	76.80
	16	54.75	80.70	4.00	14.00	53.00	82.40	50.63	77.64
	32	57.91	81.33	4.00	14.00	55.79	83.26	53.47	78.37

Table 16: Additional cross-backbone Whole-rollout controls. Qwen3 rows use the September 10, 2026 report; Llama uses the complete 2-epoch evaluation supplied subsequently. Each cell is a percentage; All / 832 is the whole-workload result. Bold marks the highest reported value in each metric column across all models and budgets in this table, including ties.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
Qwen3-4B + SFT + RL (Whole-rollout)	1	45.25	58.23	2.00	2.00	44.21	55.36	42.07	53.25
	4	53.48	65.19	2.00	4.00	54.08	64.59	50.72	61.18
	8	57.28	68.04	2.00	6.00	57.30	67.60	53.97	64.06
	16	60.44	70.57	2.00	6.00	59.87	69.74	56.61	66.23
	32	61.71	71.84	2.00	6.00	61.80	71.03	58.17	67.43
Qwen3-14B + SFT + RL (Whole-rollout)	1	50.32	67.09	0.00	2.00	40.77	55.15	41.95	56.49
	4	56.65	70.89	2.00	4.00	51.72	64.38	50.60	63.22
	8	62.03	75.95	2.00	6.00	55.36	69.10	54.69	67.91
	16	64.56	77.53	2.00	6.00	58.58	72.10	57.45	70.19
	32	65.51	79.43	2.00	8.00	61.80	74.03	59.62	72.12
Llama 3.1-8B + SFT + RL (Whole-rollout)	1	46.20	57.59	2.00	6.00	43.13	54.72	41.83	52.88
	4	58.23	71.20	4.00	10.00	55.58	66.95	53.49	65.14
	8	62.34	74.37	8.00	12.00	60.09	71.67	57.81	69.11
	16	66.46	76.27	8.00	12.00	65.24	74.46	62.26	71.39
	32	70.57	77.53	8.00	12.00	68.67	76.82	65.75	73.20

K SUPPLEMENTARY EXPERIMENTS AND ANALYSES

Each experiment below asks a concrete question and analyzes the corresponding results. Appendix J retains the uniform raw-data tables; the analyses here explain what the comparisons reveal.

K.1 CAN FAILED RESPONSES STILL SUPPLY A SUCCESSFUL PROOF?

Figure 6 compares pass and compose for six Bare models; Table 9 gives the numerical results. At $k = 1$, processing acts within one response; larger budgets also allow cross-response composition. This evaluates the full inference framework, including both filtering and composition.

1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997

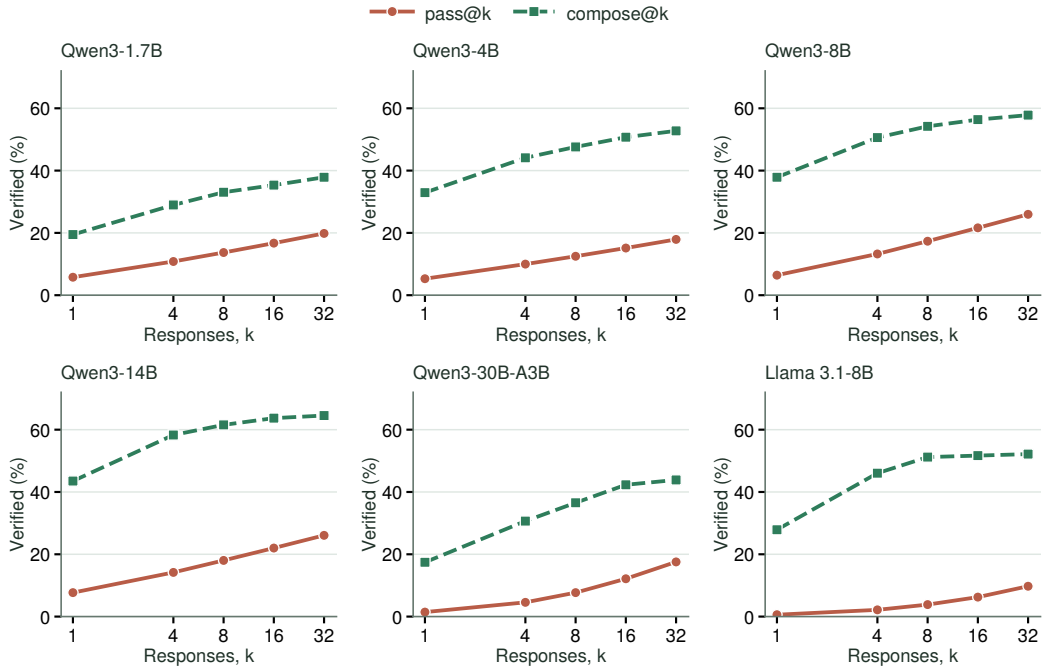


Figure 6: Bare-model probe curves for all six models. Each panel compares $\text{pass}@k$ and $\text{compose}@k$; complete numerical results are in Table 9.

Useful clauses are already recoverable at a one-response budget: Qwen3-8B reaches 37.86% $\text{compose}@1$ versus 6.43% $\text{pass}@1$, and Llama 3.1-8B reaches 27.88% versus 0.60%. Additional sampling helps, but the final budget doubling yields smaller compositional gains. From 16 to 32 responses, compose rises from 56.37% to 57.81% for Qwen3-8B and from 51.68% to 52.16% for Llama, while pass rises from 21.63% to 25.96% and from 6.25% to 9.74%, respectively. These curves show diminishing returns within the tested range; the asymptotic ceiling remains unknown.

K.2 CAN BETTER STANDALONE ANSWERS MAKE COMPOSITION WORSE?

We compare Bare, SFT (single), and composed-target SFT on the same 832 programs. The two SFT datasets contain approximately equal numbers of training examples. Single targets are independently filtered responses; composed targets pool discoveries across responses.

SFT (single) improves pass over Bare but lowers compose at every reported budget. Composed-target SFT exceeds both models on both metrics: relative to SFT (single), its compose gains are 40.70 points at $k = 1$ and 30.18 at $k = 32$, with pass gains of 22.00 and 24.76 points. Thus better standalone answers need not supply better compositions; supervision from broader candidate pools improves both in this comparison. Main-text Figure 4 shows compose , and Figure 7 shows pass ; per-stratum data are in Tables 10 and 11.

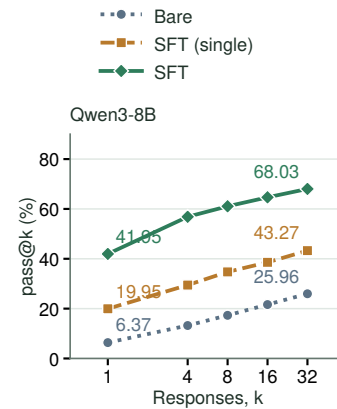


Figure 7: Qwen3-8B SFT targets and $\text{pass}@k$. Labels mark @1 and @32.

K.3 REWARD ABLATION: SHOULD ONE BAD CLAUSE CANCEL ALL CREDIT?

Reward configuration. The reported RL runs apply the per-response admission cap without penalizing clauses beyond that cap, consistently with the reward definition above.

The reward variants use different correctness gates for Equation 8. *Binary* assigns one iff the processed response is nonempty and every clause survives Houdini. *Whole-rollout* pays the strength score $\text{cov}(V_i)$ only when every processed clause survives,

$$r_i^{\text{whole}} = \mathbf{1}[V_i = \text{Unique}(\text{First}_{m_{\max}}(\text{parts}(y_i)))] \frac{|R_i|}{|\mathcal{N}|}.$$

Clause-decomposed removes this all-or-nothing indicator and scores each response by the coverage of its surviving V_i . Full (default) adds response-level Shapley credit: $r_i^{\text{Full}} = r_i^{\text{Clause-decomposed}} + 0.3\phi_i$, with the fixed survivor coverage game defined in Appendix A.6.1. All four variants admit at most $m_{\max}$ clauses per response and include no separate redundancy or overflow penalty.

Table 17: Available reward ablations across backbones on all 832 programs (%). Full after SFT uses the main 69.23% evaluation throughout; exploration diagnostics are discussed in Appendix K.9. Qwen3-8B reward variants are reported in Tables 14 and 15; additional Whole-rollout controls are in Table 16, and the other-backbone Full results are in Table 13. Dashes indicate unavailable Qwen3-14B Clause-decomposed measurements.

Reward	pass@k					compose@k				
	1	4	8	16	32	1	4	8	16	32
<i>Qwen3-8B, RL directly from Bare</i>										
Binary	4.45	4.45	4.45	4.45	4.45	4.55	4.55	4.55	4.55	4.55
Whole-rollout	15.50	16.59	16.83	17.67	18.39	20.79	21.88	22.24	22.84	23.32
Clause-decomposed	9.01	15.87	18.87	20.31	21.63	48.80	57.69	60.70	63.70	66.35
Full (default)	2.88	5.41	7.81	9.62	11.06	51.80	60.46	64.18	68.87	70.67
<i>Qwen3-8B, RL after SFT</i>										
Binary	8.77	11.30	13.22	14.30	15.50	7.09	10.82	13.34	14.54	15.99
Whole-rollout	38.82	45.19	48.68	50.84	53.73	43.63	50.12	53.37	56.01	58.53
Clause-decomposed	35.94	51.44	54.33	57.69	61.30	67.91	75.84	76.80	77.64	78.25
Full (default)	30.78	43.44	47.47	50.63	53.47	69.23	75.24	76.80	77.64	78.37
<i>Qwen3-4B, RL after SFT</i>										
Whole-rollout	42.07	50.72	53.97	56.61	58.17	53.25	61.18	64.06	66.23	67.43
Full (default)	35.10	47.12	53.25	58.17	62.74	67.55	75.60	78.00	78.85	79.21
<i>Qwen3-14B, RL after SFT</i>										
Whole-rollout	41.95	50.60	54.69	57.45	59.62	56.49	63.22	67.91	70.19	72.12
Clause-decomposed	–	–	–	–	–	–	–	–	–	–
Full (default)	35.82	48.80	53.00	55.53	58.05	69.35	75.12	76.44	77.64	78.25
<i>Llama 3.1-8B, RL after SFT</i>										
Whole-rollout	41.83	53.49	57.81	62.26	65.75	52.88	65.14	69.11	71.39	73.20
Full (default)	35.70	48.32	53.00	56.97	59.86	61.90	72.00	74.40	75.24	75.72

Preserving partial contributions improves composition. From Bare Qwen3-8B, Whole-rollout raises pass@1 from 6.43% to 15.50% but lowers compose@32 from 57.81% to 23.32%. Clause-decomposed restores 66.35% compose@32 by crediting surviving clauses even when their response fails. Its advantage over Whole-rollout is 43.03 points from Bare and 19.72 points after SFT at $k = 32$, supporting clause-level credit under both initializations. Binary reaches only 4.55% and 15.99%, respectively.

Full also exceeds Whole-rollout after SFT on Qwen3-4B and Llama 3.1-8B, by 14.30 and 9.02 points at compose@1. These cross-backbone rows compare the complete reward against Whole-rollout; they do not isolate the added Shapley term. Exploration diagnostics for Full and Clause-decomposed are in Appendix K.9.

K.3.1 DOES TRAINING REWARD TRACK VERIFICATION QUALITY?

The supplied Qwen3-8B training records cover 2 RL epochs for four rewards under Bare and SFT initialization, giving eight complete curves. Figure 8 reports the mean reward at each training step for each configuration, as confirmed for the supplied CSV.

Binary reward saturates without strong verification.

In both initializations, every recorded Binary reward in the final 20 updates is 1.000. Yet the corresponding two-epoch evaluations reach only 4.55% compose@32 from Bare and 15.99% after SFT (Table 17). Saturating the correctness reward therefore does not ensure invariants strong enough to prove hidden targets.

Reward growth must be judged against verification.

Full, Clause-decomposed, and Whole-rollout all end with higher rewards than they start with under both initializations. Their upward trajectories, alongside Binary’s saturation, show successful optimization of different signals; the endpoint compose results determine whether that progress produces useful invariants. Absolute heights cannot rank the configurations: the correctness gates differ, and Full adds weighted Shapley credit.

The CSV records per-step mean rewards. Per-group reward variance is required to measure how often Full prevents group reward collapse. The mechanism is formalized in Appendix A.7.1, and additional exploration diagnostics are analyzed in Appendix K.9.

K.4 CAN MORE CLAUSES HURT ANSWERS YET HELP PROOFS?

We vary the per-response clause cap for Bare, SFT, and SFT+RL Qwen3-8B. Within each model, all caps reuse the same saved response pool and evaluation settings: only the first c parsed clauses of each response are admitted to standalone verification or pooled filtering. This is inference-time truncation of saved responses. The sweep is a separate evaluation; its cap-16 endpoints are not substituted for the main-result models.

Table 18: Qwen3-8B clause-cap ablation (%). Within each model, all rows reuse the same saved responses. Pass values use the nearest-integer projection of supplied rates described in Appendix H; this uses the reported rates directly. Compose values retain the supplied results. Bold marks each column’s maximum across caps.

Cap	pass@1			compose@1			compose@8		
	Bare	SFT	SFT+RL	Bare	SFT	SFT+RL	Bare	SFT	SFT+RL
1	12.14	19.59	22.84	11.06	19.23	22.72	17.67	35.10	25.48
2	9.50	27.28	30.29	17.31	30.89	34.01	25.24	49.40	39.18
4	8.17	41.47	45.31	27.76	45.67	50.48	40.75	67.07	57.45
8	7.09	44.47	41.95	35.46	57.09	63.34	52.40	73.56	71.39
16	6.61	42.79	30.89	39.42	64.90	69.11	54.21	76.56	76.32

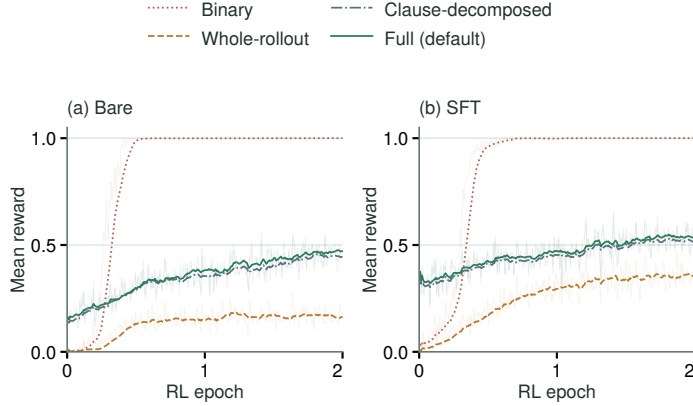


Figure 8: Mean training rewards under (a) Bare and (b) SFT initialization. Pale lines show the per-step means; emphasized lines are trailing 15-step means, using available steps at the start. Reward scales differ across objectives; parentheses mark the default.

The best clause budget depends on the success criterion. Bare pass@1 is highest at cap 1 and falls as the cap grows. SFT and SFT+RL instead peak at caps 8 and 4, respectively. For SFT+RL, increasing the cap from 4 to 16 lowers displayed pass@1 from 45.31% to 30.89%, while compose@1 rises from 50.48% to 69.11% and compose@8 from 57.45% to 76.32%. Both compose metrics increase monotonically with the cap for all three models in this sweep. Additional clauses can therefore hurt complete-response verification while expanding the candidates available for filtering; these aggregates do not distinguish invalid clauses from prover timeouts or other causes of standalone failure.

Does Better Composition Require More Distinct Clauses? Table 19 separates response length from the number of distinct and surviving clauses across eight responses.

Table 19: Structural statistics for the same clause-cap sweep. Clauses is the mean admitted clause count per response; Unique@8 is the mean number of canonical clause keys across eight responses; Survivors@8 is the mean number retained after filtering their pooled union. These are mean counts.

Model	Statistic	Cap 1	Cap 2	Cap 4	Cap 8	Cap 16
Bare	Clauses	1.00	1.98	3.88	7.24	12.37
	Unique@8	2.72	5.50	11.67	24.30	41.69
	Survivors@8	1.42	2.49	5.37	11.33	19.91
SFT	Clauses	1.00	2.00	3.96	7.52	12.73
	Unique@8	3.37	6.69	14.26	30.14	56.07
	Survivors@8	2.92	5.71	12.10	25.27	46.35
SFT+RL	Clauses	1.00	2.00	4.00	7.98	14.03
	Unique@8	1.41	2.90	6.41	15.90	35.80
	Survivors@8	1.18	2.33	5.06	12.06	26.07

At cap 16, Bare and SFT admit similar numbers of clauses per response (12.37 versus 12.73), but SFT supplies more unique clauses (56.07 versus 41.69) and more than twice as many survivors (46.35 versus 19.91). Output length alone therefore does not explain the SFT gain.

Post-SFT RL shows a different pattern. It admits more clauses per response than SFT (14.03 versus 12.73), yet produces fewer unique clauses across eight responses (35.80 versus 56.07) and fewer survivors (26.07 versus 46.35). Nevertheless, compose@1 is 4.21 points higher, while compose@8 is nearly unchanged (76.32% versus 76.56%). Thus its one-response gain coexists with a smaller distinct candidate pool, consistent with useful candidates becoming easier to obtain at a small sampling budget. This comparison does not support an increase in cross-response diversity; clause counts alone also do not measure logical strength or necessity.

K.5 DO STRUCTURED NEGATIVES MAKE COVERAGE MORE INFORMATIVE?

Does structured sampling improve trained-model verification? The negative-sampler ablation uses matched Qwen3-8B runs from the Bare initialization under the Full reward that differ only in the sampler. These archived runs form a separate experiment; their Structured evaluation differs from the main Full evaluation. Structured is the sampler of Section 4.2 (relation and post-exit families); Random draws uniform local perturbations under the same trace budget.

Table 20: Negative-sampler ablation on Qwen3-8B, trained from the Bare initialization under the full reward (%). Bold marks the better sampler per column.

Negative sampler	pass@k					compose@k				
	1	4	8	16	32	1	4	8	16	32
Random	7.21	11.78	13.82	15.87	18.03	46.88	55.41	58.77	60.82	62.26
Structured (default)	7.09	12.86	15.63	18.39	20.79	48.92	59.62	63.34	66.11	67.31

Structured sampling improves compose@k by 2.0–5.3 points at every budget. Pass@k stays within −0.1 to 2.8 points of Random.

K.5.1 CAN A TARGET-HIDDEN SCORE PREDICT TARGET VERIFICATION?

This diagnostic compares the two scores on fixed candidates, separately from the RL ablation above. The population contains 5,711 candidate sets from 691 benchmark programs with both accepted and rejected sets, including archived target-hidden GPT-5-nano tool outputs and one verified reference set per program. The remaining 141 programs lack both classes for within-program ranking.

Both samplers score exactly the same candidates, frozen Houdini survivors, positive states, and final verification labels. They use the target-hidden program, seed 0, and a budget of 60 negative traces; all 691 programs yield random negatives. Hidden targets supply only final Frama-C/WP labels, never scores. AUROC is computed within each program and macro-averaged. Confidence intervals use 5,000 paired bootstrap resamples over programs.

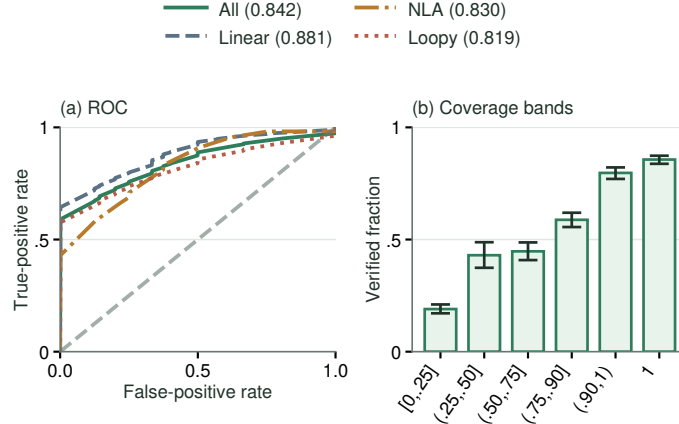


Figure 9: Structured coverage on 5,711 candidate sets. (a) Macro ROC and AUROC. (b) Verification by coverage band, with Wilson 95% intervals.

Population	Programs	Candidates	Structured AUROC (default)	Random AUROC
Linear	248	2,173	0.881	0.849
NLA	49	439	0.830	0.798
Loopy	394	3,099	0.819	0.791
All	691	5,711	0.842	0.812

Structured coverage achieves a macro AUROC of 0.842 (program-level bootstrap 95% CI: [0.826, 0.858]), compared with 0.812 for Random ([0.797, 0.828]). The paired Structured-minus-Random difference is 0.030, with a 95% bootstrap CI of [0.019, 0.041]. Structured coverage has higher macro AUROC in all three categories. Both scores show alignment with final verification, and structured sampling improves the ranking of sufficient versus insufficient invariant sets on this fixed population. Coverage is an empirical diagnostic; final verification and generation determine whether a sufficient invariant is found.

This population consists of archived tool outputs and verified references. Training-time response groups require separate evidence of informative within-group reward differences.

K.6 WHEN DOES FILTERING TOGETHER BEAT FILTERING SEPARATELY?

We compare filtering each response before pooling (pre-filtering) with one joint filter over the pooled clauses (post-filtering), using the same saved Qwen3-8B response prefixes on 832 programs. CRAFT uses the latter order in inference and training; the operator and its theoretical properties are defined in Appendix A.

Joint filtering approaches the same verification rate at much lower cost. With the 900-second cap, post-filtering reaches 64.2% compose@32 versus 64.4% for pre-filtering (Table 21). The recorded whole-workload cost is 127 versus 1,472 CPU-hours, or about 8.6% as much. These values are total CPU-hours for the reported 832-program evaluation workload. The main observed advantage is avoiding repeated filtering calls while retaining nearly the same verification rate.

A tight timeout can erase the benefit of a larger candidate pool. At a 300-second cap, post-filtering costs 113 CPU-hours but reaches only 58.7% compose@32. Increasing the cap to 900

Table 21: Pre-filtering and post-filtering for invariant verification on the 832-program benchmark (Qwen3-8B, saved prefixes up to 32 rollouts; compose@ k in %). CPU-h is the whole-workload total.

Method	Subset	@1	@4	@8	@16	@32	CPU-h
Post-filter (300 s)	All	40.4	52.8	56.0	58.1	58.7	113
	Loopy	43.3	56.9	60.9	63.1	63.7	
	Linear	41.1	53.8	56.3	57.9	58.5	
	NLA	8.0	8.0	8.0	12.0	12.0	
Post-filter (900 s)	All	40.4	53.4	58.4	62.5	64.2	127
	Loopy	43.3	57.7	63.3	68.7	70.2	
	Linear	41.1	54.1	59.2	61.4	63.3	
	NLA	8.0	8.0	8.0	12.0	14.0	
Pre-filter	All	40.4	52.8	57.9	62.1	64.4	1472
	Loopy	43.3	57.3	62.9	68.0	71.0	
	Linear	41.1	53.2	58.5	61.4	63.0	
	NLA	8.0	8.0	8.0	12.0	12.0	

seconds recovers 5.5 points with 14 additional CPU-hours. Thus the cost advantage depends on allowing enough time for the larger pooled obligations; the ideal inclusion property does not guarantee higher verification under finite prover budgets.

K.7 HOW DOES TARGET VISIBILITY CHANGE VERIFICATION AND SAMPLING NEEDS?

We expose the target assertions and postconditions during generation for the same Qwen3-8B SFT+RL model, holding other settings and response budgets fixed. The model was trained with targets hidden; neither condition adds training or iterative verifier feedback. The hidden reference matches Table 13.

Target visibility improves both whole-workload metrics at every tested budget (Figure 10). Compose gains 1.32 points at $k = 1$ and 3.12–3.73 points at $k \geq 4$, corresponding to 11 and 26–31 additional verified programs. Pass gains range from approximately 1.68 points at $k = 1$ to 7.21 at $k = 32$.

Visible compose@4 reaches 78.73%, compared with 78.37% for hidden compose@32. Target visibility therefore reduces the sampling budget in this evaluation. The saved-pool estimates and prefix compositions remain subject to sampling variance (Appendix L).

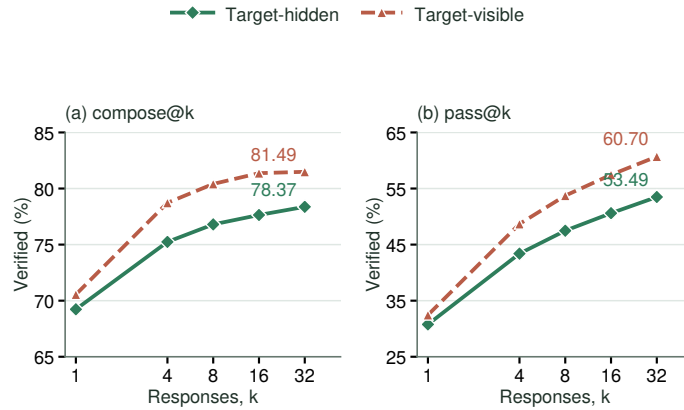


Figure 10: Target visibility for the same Qwen3-8B SFT+RL model. (a) compose@ k ; (b) pass@ k . No retraining or iterative feedback.

Table 22: CRAFT with target-visible (target-vision) and target-hidden inference using the same Qwen3-8B SFT+RL model, by benchmark stratum and sampling budget. Each cell reports a percentage; All / 832 is the whole-workload result. Visible compose rates are computed from exact success counts; visible pass rates retain the reported one-decimal precision. Hidden results retain the paired evaluation used in this experiment.

Configuration	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
CRAFT (target-hidden)	1	32.91	75.00	2.00	14.00	32.40	71.24	30.78	69.23
	4	46.52	79.75	4.00	14.00	45.71	78.76	43.44	75.24
	8	50.95	80.70	4.00	14.00	49.79	80.90	47.47	76.80
	16	54.75	80.70	4.00	14.00	53.00	82.40	50.63	77.64
	32	57.91	81.33	4.00	14.00	55.79	83.26	53.47	78.37
CRAFT (target-vision)	1	34.49	73.42	4.00	10.00	34.12	75.11	32.45	70.55
	4	51.58	80.70	6.00	14.00	51.50	84.33	48.68	78.73
	8	56.65	82.59	6.00	14.00	56.87	86.05	53.73	80.41
	16	60.76	83.23	6.00	14.00	60.94	87.34	57.45	81.37
	32	64.24	83.23	6.00	14.00	64.16	87.55	60.70	81.49

Table 22 shows that the composition gains vary by stratum. Loopy improves at every budget, by 3.87–5.57 percentage points. Linear improves for $k \geq 4$, but its compose@1 falls from 75.00% to 73.42%. NLA compose@1 falls from 14.00% to 10.00%, while both conditions reach 14.00% for $k \geq 4$. The target-visibility gain is concentrated in Loopy and is absent or negative for NLA.

The results isolate the effect of target visibility without iterative refinement.

K.8 DOES THE ADVANTAGE SURVIVE CHANGES IN MODELS AND TOOLS?

We examine generator choice, verification cost with a fixed model, and how tool interfaces and feedback conditions affect the comparison.

K.8.1 DOES THE BEST STANDALONE MODEL ALSO COMPOSE BEST?

Table 23 holds target hiding and reasoning-disabled generation fixed across four API models. Claude Sonnet-4.6 leads at compose@1 (56.97%), while DeepSeek V4-Flash leads at compose@8 (74.40%). At eight responses, GPT-5 has higher pass than DeepSeek (51.68% versus 48.20%) but lower compose (72.00% versus 74.40%). Choosing a generator by standalone success can therefore miss the best generator for composition.

Table 23: Complete reasoning-disabled cross-model results by benchmark stratum and sampling budget. Each cell reports a percentage; All / 832 is the whole-workload result. Rows for each model come from one archived rollout pool of 10 responses per task. Pass@ k uses a projected estimate from the saved response pool; compose@ k evaluates the first k saved responses. Bold marks one maximum per metric column across all models and budgets; ties are marked only at their first occurrence. GPT-5-mini is excluded because it has no reasoning-disabled setting.

Model	k	Linear / 316		NLA / 50		Loopy / 466		All / 832	
		pass	comp	pass	comp	pass	comp	pass	comp
GPT-5-nano	1	3.80	34.81	0.00	22.00	3.86	34.55	3.61	33.89
	4	10.76	48.10	2.00	30.00	11.37	51.72	10.58	49.04
	8	15.82	55.38	4.00	44.00	17.60	56.87	16.11	55.53
GPT-5	1	35.13	52.22	6.00	36.00	33.69	52.36	32.57	51.32
	4	47.78	64.87	8.00	52.00	50.64	67.38	47.00	65.50
	8	51.90	70.25	10.00	66.00	56.22	73.82	51.68	72.00
Claude Sonnet-4.6	1	34.49	55.38	8.00	44.00	40.77	59.44	36.30	56.97
	4	40.19	62.97	8.00	52.00	47.21	66.31	42.31	64.18
	8	42.72	66.46	10.00	64.00	50.21	67.60	44.95	66.95
DeepSeek V4-Flash	1	28.80	52.22	6.00	44.00	30.04	51.72	28.13	51.44
	4	42.41	66.46	8.00	60.00	47.64	69.96	43.27	68.03
	8	46.52	71.52	8.00	72.00	53.65	76.61	48.20	74.40

Pass displays nearest-integer projections of archived pool estimates; compose uses saved prefixes (Appendix H).

K.8.2 WHAT VERIFICATION-COST TRADEOFF DOES A FIXED MODEL ACHIEVE?

With GPT-5-nano and target hiding fixed, CRAFT compose@4 verifies 49.04% of programs using 1,905.10 tokens per task, compared with AutoSpec’s 42.19% and 2,181.72 tokens. AutoSpec itself pools eight parallel proposals, so this comparison evaluates the complete clause-processing pipelines.

SESpec iteratively repairs a running candidate and verifies 29.69% using 22,081.61 tokens per task. CRAFT compose@1 verifies 33.89% with 1,136.82 tokens, about 19.4 times fewer. This comparison supports clause reuse under limited target feedback; the native pipelines also differ in extraction and repair, so it does not isolate orchestration as the only cause.

Figure 11 adds measured end-to-end time to the token comparison. A separate local cost-run model reaches 70.31%, 75.60%, and 77.28% at compose@1, @4, and @8 in 28.2, 33.5, and 40.9 seconds per task. At @4 and @8 it verifies more programs in less recorded time than GPT-5-nano CRAFT (46.40 and 69.99 seconds). This model differs from the selected main result. Times reflect each method’s native serving setup; GPT-5-nano CRAFT combines archived request latency with measured filtering and final verification.

Table 24 reports all results; the parameter settings are in Table 25. Daikon incurs verification time but uses no model tokens. Tool-specific applicability and extraction limitations are discussed immediately below the tables.

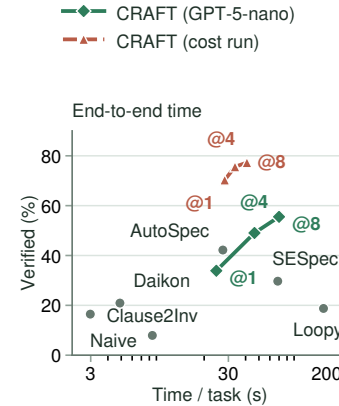


Figure 11: Verification versus time under native serving setups. Daikon uses no model tokens.

Table 24: Complete target-hidden tool comparison underlying Figure 2: the reasoning-disabled rows alongside the archived medium-reasoning rerun; Daikon makes no model call and is listed once. Each cell reports the verification rate for Linear, NLA, Loopy, or all 832 programs. Medium-effort results are available only for the single-response measurements. The reasoning-disabled compose@1, compose@4, and compose@8 rows are prefix compositions of the same archived ten-rollout pool.

Method	Reason.	Linear	NLA	Loopy	All	Tokens/task	Time/task
AutoSpec	none	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
AutoSpec	medium	65.82%	8.00%	64.81%	61.78%	25,154.54	54.77 s
SESpec	none	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
SESpec	medium	56.96%	6.00%	49.57%	49.76%	44,807.05	117.31 s
Clause2Inv	none	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Clause2Inv	medium	19.94%	2.00%	0.00%	7.69%	385.80	29.04 s
Daikon	—	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	none	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Loopy	medium	72.78%	12.00%	70.82%	68.03%	111,973.15	381.01 s
Naive	none	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
Naive	medium	48.73%	8.00%	47.42%	45.55%	4,493.37	28.87 s
CRAFT (pass@1)	none	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (pass@1)	medium	61.08%	14.00%	54.08%	54.33%	6,928.30	53.52 s
CRAFT (compose@1)	none	34.81%	22.00%	34.55%	33.89%	1,136.82	24.52 s
CRAFT (compose@1)	medium	64.24%	14.00%	63.09%	60.58%	6,928.30	66.60 s
CRAFT (compose@4)	none	48.10%	30.00%	51.72%	49.04%	1,905.10	46.40 s
CRAFT (compose@8)	none	55.38%	44.00%	56.87%	55.53%	2,929.47	69.99 s

Table 25: Sampling parameters used in the reasoning-disabled target-hidden tool comparison.

Method	Sampling budget / task	Temperature / top- <i>p</i>	Completion cap	Reasoning
AutoSpec	8	1.0 / 1.0	8,192	none
SESpec	1 initial + up to 3 refinements per phase	1.0 / 1.0	8,192	none
Clause2Inv	adaptive; 1 per call	1.0 / 1.0	8,192	none
Daikon	no model call	—	—	—
Naive	1	1.0 / 1.0	8,192	none
CRAFT	1, 4, or 8	1.0 / 1.0	8,192	none
Loopy	15 initial + up to 7 repairs	0.7 / 1.0	8,192	none

Token means cover tasks with at least one model call; verification rates always retain all 832 tasks. AutoSpec’s token count is the native estimate for one request returning eight proposals, averaged over the 828 tasks with records; four failed tasks have no token record.

K.8.3 WHICH INTERFACE AND FEEDBACK CONDITIONS EXPLAIN TOOL DIFFERENCES?

Applicability affects the benchmark denominator. Clause2Inv normally derives false-state counterexamples from the postcondition, which is unavailable under target hiding. Its transition verification conditions are also unavailable for the 466 Loopy programs; these remain failures in the common denominator. Its low aggregate rate therefore reflects both applicability limits and candidate-generation performance.

Extraction can prevent generated annotations from reaching verification. Loopy’s initial responses contain invariant annotations in 98.8% of cases, but its native extractor passes only 1.75% of initial responses to Houdini. These interface measurements help explain why generating annotations does not necessarily produce verifier-consumable candidates.

Reasoning settings can change the ranking. In the archived medium-reasoning runs, AutoSpec rises from 42.19% to 61.78% and Loopy from 18.75% to 68.03%, reversing their ranking. CRAFT compose@1 rises from 33.89% to 60.58%, while Clause2Inv changes from 7.93% to 7.69%. These runs use each system’s reported reasoning condition and different token costs; they do not hold the

token budget fixed. Tool comparisons should therefore identify the generator, reasoning setting, applicability, and extraction pipeline alongside the final rate.

K.9 SHAPLEY ANALYSIS: DOES GROUP CREDIT HELP EXPLORATION?

We compare Full with Clause-decomposed on Qwen3-8B to examine when added group credit helps from Bare versus SFT after 2 RL epochs.

At equal coverage, Full gives more credit to traces rejected by fewer responses, encouraging complementary effective coverage. The allocation and its Shapley equivalence are defined in Appendix A.6.1.

Table 26 compares Clause-decomposed and Full on Qwen3-8B after 2 RL epochs, with structured negative sampling and 9,024 training programs. Full after SFT uses the main evaluation at 69.23% compose@1, consistently with Tables 13 and 17.

Table 26: Clause-decomposed versus Full for Qwen3-8B after 2 RL epochs (%). Full is the default reward; the post-SFT Full row uses the main evaluation. Bold marks the better reward within each initialization and metric column, including ties.

Reward	pass@ k					compose@ k				
	1	4	8	16	32	1	4	8	16	32
<i>RL from Bare</i>										
Clause-decomposed	9.01	15.87	18.87	20.31	21.63	48.80	57.69	60.70	63.70	66.35
Full (default)	2.88	5.41	7.81	9.62	11.06	51.80	60.46	64.18	68.87	70.67
<i>RL after SFT</i>										
Clause-decomposed	35.94	51.44	54.33	57.69	61.30	67.91	75.84	76.80	77.64	78.25
Full (default)	30.78	43.44	47.47	50.63	53.47	69.23	75.24	76.80	77.64	78.37

From Bare, Full improves composition throughout the tested budgets. Full improves compose by 2.77–5.17 points across the tested budgets, including 48.80% to 51.80% at $k = 1$ and 66.35% to 70.67% at $k = 32$. Its advantage remains substantial at the largest measured budget, while pass decreases by 6.13–11.06 points: better composition can accompany less successful complete responses.

After SFT, the added benefit is small. Full changes compose by -0.60 to $+1.32$ points: 67.91% to 69.23% at $k = 1$, and 78.25% to 78.37% at $k = 32$. It is lower at $k = 4$ and tied at $k = 8$ and $k = 16$. Pass decreases by 5.17–8.05 points. Thus the added composition benefit after SFT is small and concentrated at the one-response budget. Composed-target SFT already supplies much of the coverage encouraged by group credit.

Does Full retain more policy entropy? The supplied training diagnostics compare Full and Clause-decomposed over 2 RL epochs. Table 27 reports an early reference measurement and the end-of-training value. Retention is computed from the displayed entropy values; these are policy statistics.

Table 27: Policy entropy under the two rewards over 2 RL epochs. Retention is final entropy divided by the early reference value. Bold marks the higher final entropy and retention within each initialization.

Initialization	Reward	Early entropy	Final entropy	Retention (%)
SFT	Full (default)	0.3195	0.1842	57.7
	Clause-decomposed	0.3233	0.1323	40.9
Bare	Full (default)	0.0588	0.0130	22.1
	Clause-decomposed	0.0773	0.0120	15.5

Full retains more policy entropy under both initializations. The absolute final-entropy difference is larger after SFT (0.1842 versus 0.1323) than from Bare (0.0130 versus 0.0120); Bare also starts from unequal reference entropies. These observations suggest that Full slows entropy loss during training.

Reward dispersion and policy entropy. The training summary reports a batch mean-to-maximum reward ratio 0.01–0.06 lower for Full than for Clause-decomposed.

Full improves $\text{compose}@k$ from Bare and retains higher policy entropy than Clause-decomposed. The Shapley mechanism and its limits are formalized in Appendix A.7.1.

L DETAILED LIMITATIONS

Sampling variance and metric aggregation. Archived $\text{pass}@k$ estimates standalone success over subsets of a saved response pool; $\text{compose}@k$ evaluates one prefix. The displayed pass values project reported estimates to integer counts; they do not recover observed verdicts or ensure additive stratum counts. Sharing the nominal budget does not pair the individual responses used in the two aggregates or equalize their variance. Reordering a fixed pool leaves its pass estimate unchanged but can alter compose . Small differences and apparent saturation can therefore reflect the sampled pool and prefix; nested budgets also make curve points correlated. Direct-count results retain the supplied run’s sampling variability. These measurements do not establish statistical significance or an asymptotic ceiling. Repeating generation and evaluating both metrics on the same sampled subsets would permit uncertainty estimates for paired differences.

Program scope. Our implementation targets numerical programs with one scalar-integer loop. The evaluation excludes nested loops and invariants over arrays, heaps, or recursively defined predicates. The negative sampler covers scalar arithmetic relations and exit boundaries (Liu et al., 2024a).

Inference engine. Local base and trained models are served with vLLM. In preliminary comparisons under nominally matched decoding settings, these models had lower verification rates with vLLM than with the other evaluated backends. API models retain their provider serving interfaces and chat templates. The reported decoding and reasoning controls are applied to each backend.

Verifier and specification language. Our filtering and all final judgments use Frama-C/WP, and candidates use ACSL. Parser behavior, generated obligations, prover integration, and syntax affect which clauses survive. The reported results characterize this verifier, specification language, and source language.

Finite candidate-pool approximation. Before Houdini, the pipeline implementation applies a lightweight AST-based filter to parse, normalize, and deduplicate candidate clauses. Clauses outside its supported ACSL subset are omitted. Each response contributes at most $m_{\max}$ clauses before global pooling; later clauses in that response are truncated, while no separate cap is applied to the pooled union.

Feedback regime. The evaluation covers target-independent training and finite-budget composition without iterative verifier-feedback optimization. In pilot runs, models reached inductive but weak invariants: the invariants passed induction, while the hidden goals left the verifier without a target-directed signal for the missing behavioral strength. The marginal gain from refinement decreased as the initial rollout pool grew.

Training variance. We did not repeat training and inference multiple times for every experimental configuration.